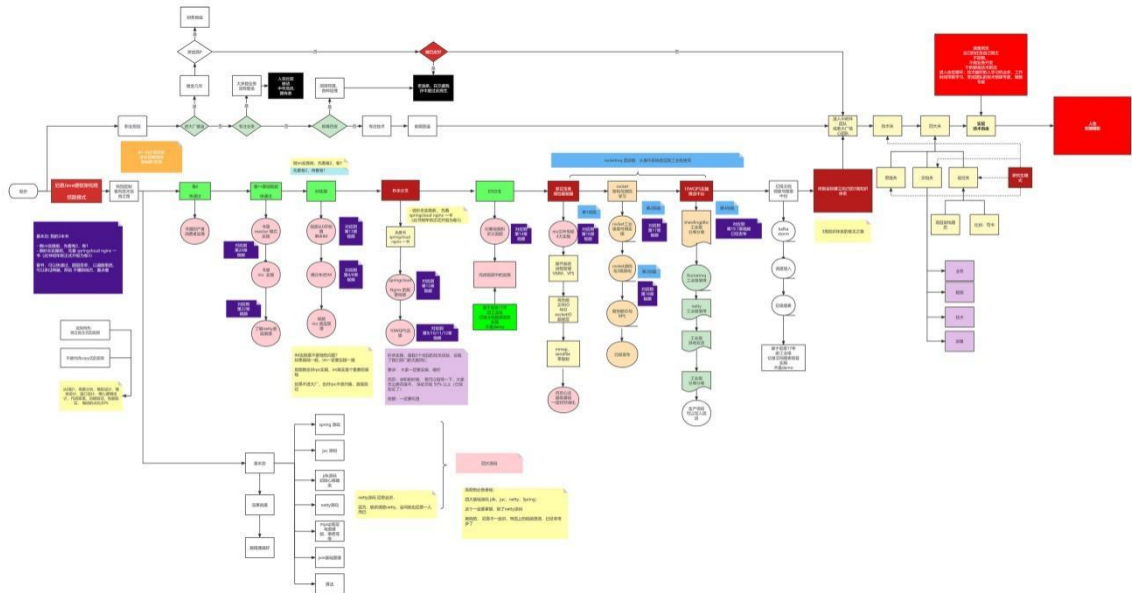


牛逼的职业发展之路

40 岁老架构尼恩用一张图揭秘：Java 工程师的高端职业发展路径，走向食物链顶端的之路

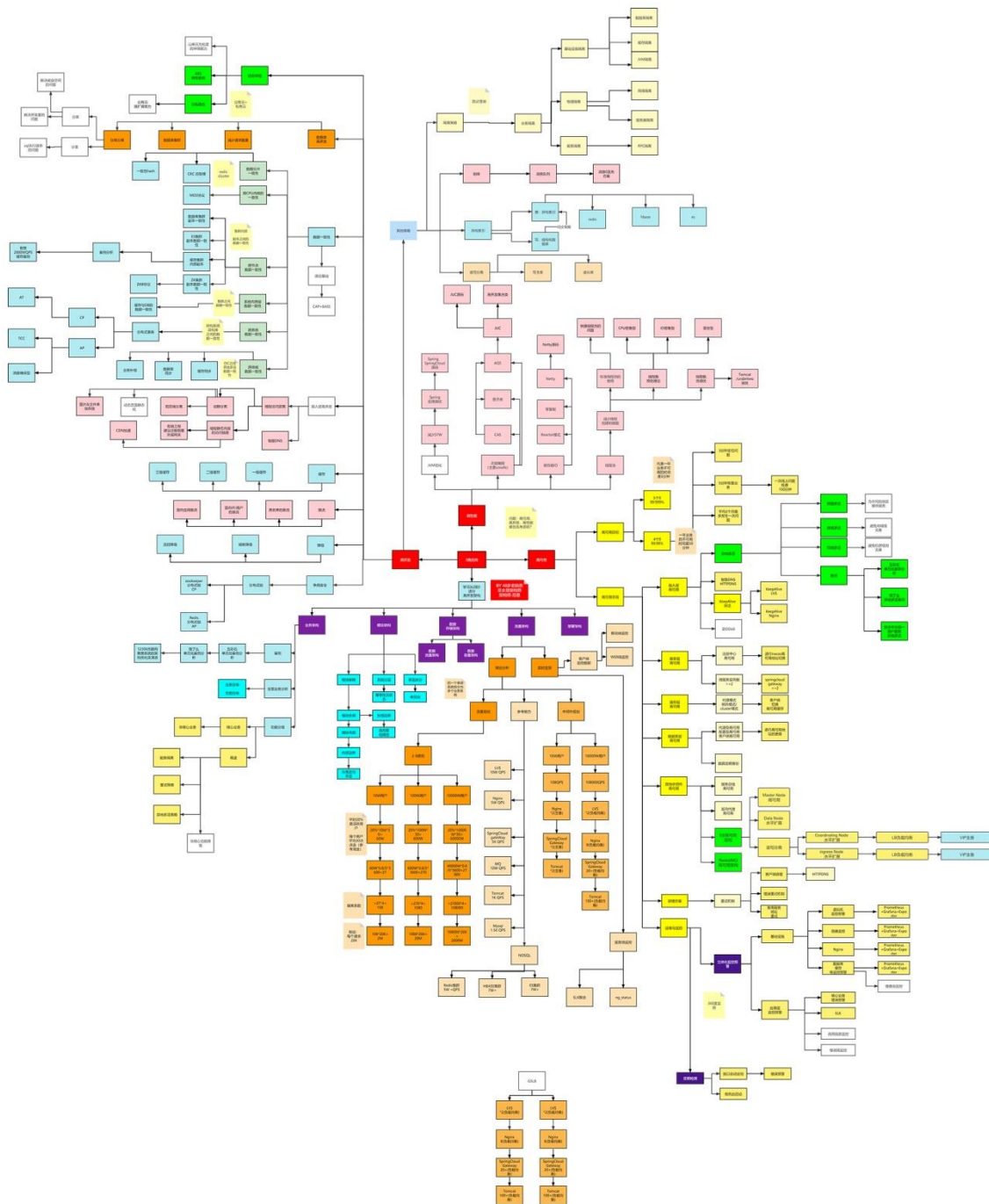
链接：<https://www.processon.com/view/link/618a2b62e0b34d73f7eb3cd7>



史上最全：价值10W的架构师知识图谱

此图梳理于尼恩的多个 3 高生产项目：多个亿级人民币的大型 SAAS 平台和智慧城市项目

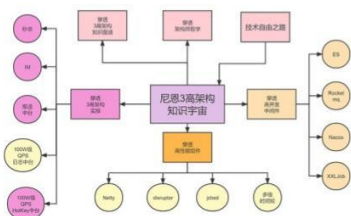
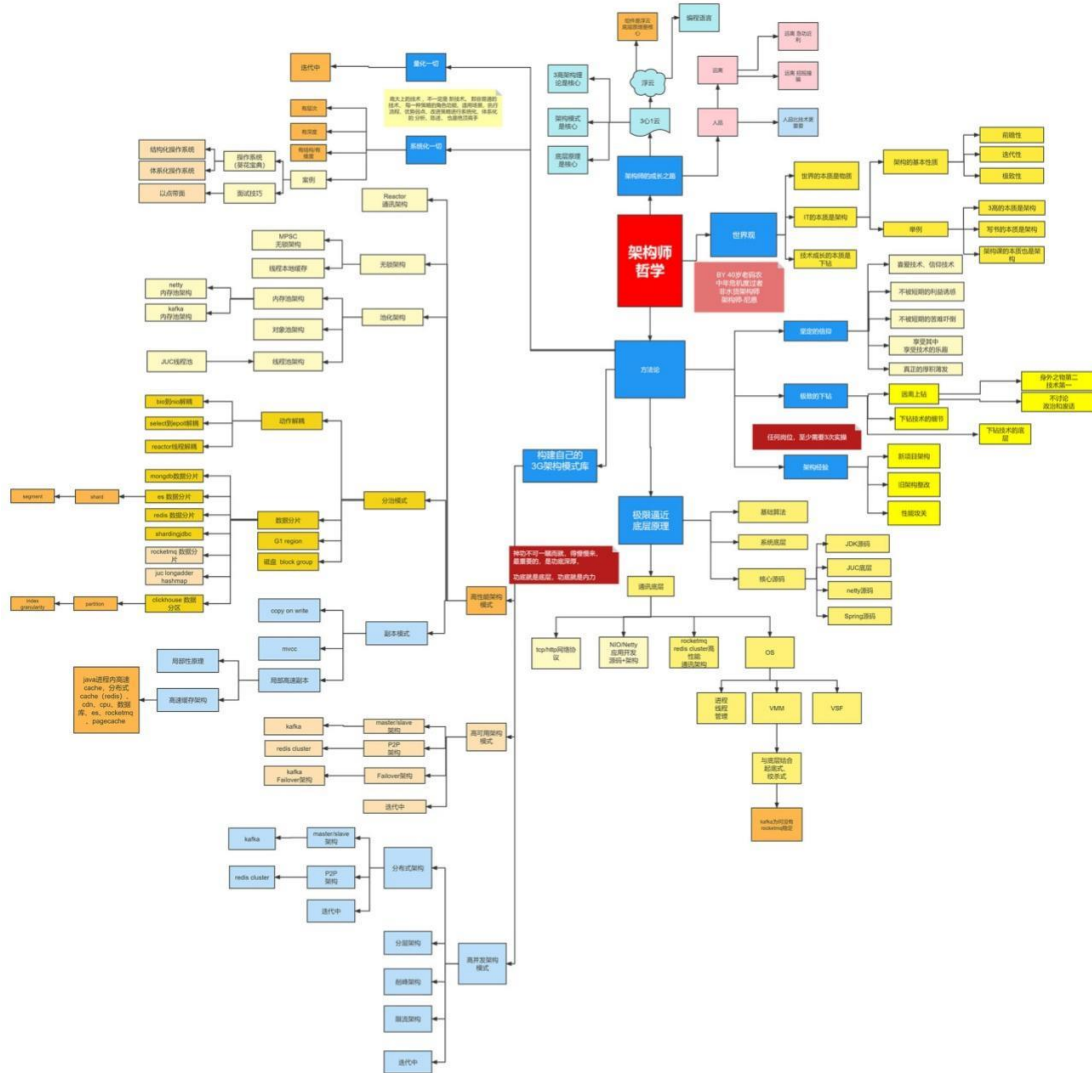
链接：<https://www.processon.com/view/link/60fb9421637689719d246739>



牛逼的架构师哲学

40 岁老架构师尼恩对自己的 20 年的开发、架构经验总结

链接: <https://www.processon.com/view/link/616f801963768961e9d9aec8>



牛逼的3高架构知识宇宙

尼恩 3 高架构知识宇宙，帮助大家穿透 3 高架构，走向技术自由，远离中年危机

链接: <https://www.processon.com/view/link/635097d2e0b34d40be778ab4>



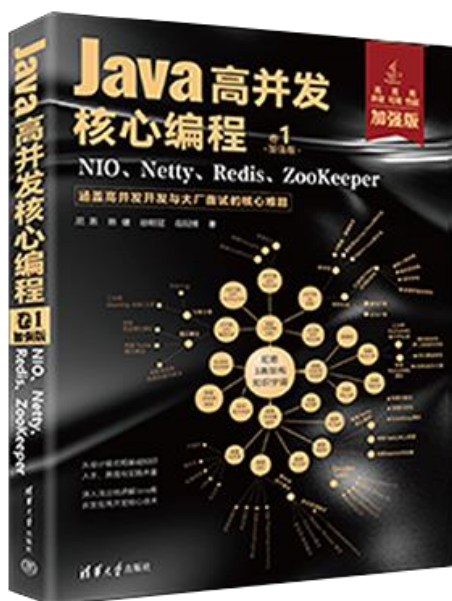
尼恩Java高并发三部曲（卷1加强版）

老版本：《Java 高并发核心编程 卷1：NIO、Netty、Redis、ZooKeeper》（已经过时，不建议购买）

新版本：《Java 高并发核心编程 卷1 **加强版**：NIO、Netty、Redis、ZooKeeper》

- 由浅入深地剖析了高并发 IO 的底层原理。
- 图文并茂地介绍了 TCP、HTTP、WebSocket 协议的核心原理。
- 细致深入地揭秘了 Reactor 高性能模式。
- 全面介绍了 Netty 框架，并完成单体 IM、分布式 IM 的实战设计。
- 详尽地介绍了 ZooKeeper、Redis 的使用，以帮助提升高并发、可扩展能力

详情：<https://www.cnblogs.com/crazymakercircle/p/16868827.html>



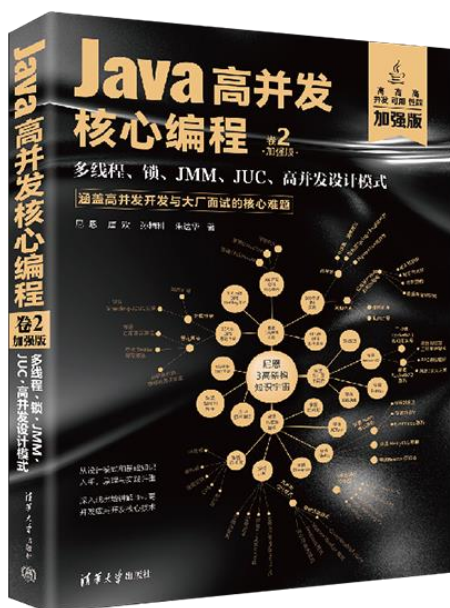
尼恩Java高并发三部曲（卷2加强版）

老版本：《Java 高并发核心编程 卷2：多线程、锁、JMM、JUC、高并发设计模式》
（已经过时，不建议购买）

新版本：《Java 高并发核心编程 卷2 **加强版**：多线程、锁、JMM、JUC、高并发设计模式》

- 由浅入深地剖析了 Java 多线程、线程池的底层原理。
- 总结了 IO 密集型、CPU 密集型线程池的线程数预估算法。
- 图文并茂的介绍了 Java 内置锁、JUC 显式锁的核心原理。
- 细致深入地揭秘了 JMM 内存模型。
- 全面介绍了 JUC 框架的设计模式与核心原理，并完成其高核心组件的实战介绍。
- 详尽地介绍了高并发设计模式的使用，以帮助提升高并发、可扩展能力

详情参阅：<https://www.cnblogs.com/crazymakercircle/p/16868827.html>



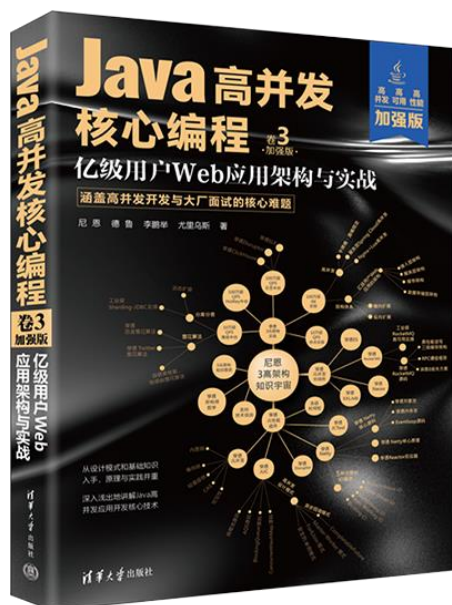
尼恩Java高并发三部曲（卷3加强版）

老版本：《SpringCloud Nginx 高并发核心编程》（已经过时，不建议购买）

新版本：《Java 高并发核心编程 卷3 **加强版**：亿级用户 Web 应用架构与实战》

- 在当今的面试场景中，3 高知识是大家面试必备的核心知识，本书基于亿级用户 3 高 Web 应用的架构分析理论，为大家对 3 高架构系统做一个系统化和清晰化的介绍。
- 从 Java 静态代理、动态代理模式入手，抽丝剥茧地解读了 SpringCloud 全家桶中 RPC 核心原理和执行过程，这是高级 Java 工程师面试必备的基础知识。
- 从Reactor 反应器模式入手，抽丝剥茧地解读了Nginx 核心思想和各配置项的底层知识和原理，这是高级 Java 工程师、架构师面试必备的基础知识。
- 从观察者模式入手，抽丝剥茧地解读了 RxJava、Hystrix 的核心思想和使用方法，这也是高级 Java 工程师、架构师面试必备的基础知识。

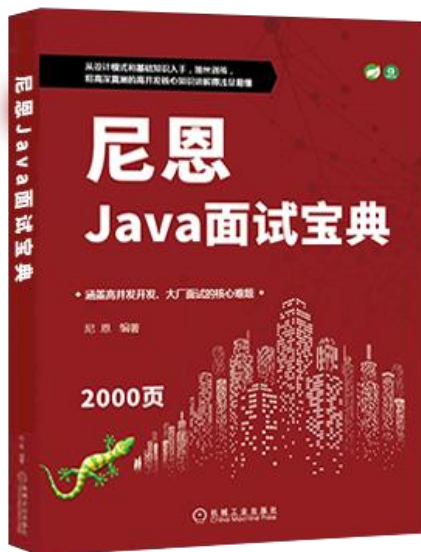
详情：<https://www.cnblogs.com/crazymakercircle/p/16868827.html>



尼恩Java面试宝典

35 个专题（卷王专供+ 史上最全 + 2023 面试必备）

详情：<https://www.cnblogs.com/crazymakercircle/p/13917138.html>



名称	
专题01: JVM面试题 (卷王专供 + 史上最全 + 2023面试必备) -V10-from-尼恩Java面试宝典-release.pdf	
专题02: Java算法面试题 (卷王专供 + 史上最全 + 2023面试必备) -V2 -from-Java面试红宝书-release.pdf	
专题03: Java基础面试题 (卷王专供+ 史上最全 + 2023面试必备) -V2-from-Java面试红宝书 -release.pdf	
专题04: 架构设计面试题 (卷王专供+ 史上最全 + 2023面试必备) -V10-from-Java面试红宝书-release.pdf	
专题05: Spring面试题_专题06: SpringMVC_专题07: Tomcat面试题 (卷王专供+ 史上最全 + 2023面试必备) -V3-from-尼恩面试宝典-release.pdf	
专题08: SpringBoot面试题 (卷王专供+ 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf	
专题09: 网络协议面试题 (卷王专供+ 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf	
专题10: TCP/IP协议 (卷王专供+ 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf	
专题11: JUC并发包与容器类 (卷王专供+ 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf	
专题12: 设计模式面试题 (卷王专供+ 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf	
专题13: 死锁面试题 (卷王专供+ 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf	
专题14: Redis 面试题 (卷王专供+ 史上最全 + 2023面试必备) -V5-from-Java面试红宝书-release.pdf	
专题15: 分布式锁 面试题 (卷王专供+ 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf	
专题16: Zookeeper 面试题 (卷王专供+ 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf	
专题17: 分布式事务面试题 (卷王专供+ 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf	
专题18: 一致性协议 (卷王专供+ 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf	
专题19: Zab协议 (卷王专供+ 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf	
专题20: Paxos 协议 (卷王专供+ 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf	
专题21: raft 协议 (卷王专供+ 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf	
专题22: Linux面试题 (卷王专供+ 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf	
专题23: Mysql 面试题 (卷王专供+ 史上最全 + 2023面试必备) -V11-from-Java面试红宝书-release.pdf	
专题24: SpringCloud 面试题 (卷王专供+ 史上最全 + 2023面试必备) -V12-from-Java面试红宝书-release.pdf	
专题25: Netty 面试题 (卷王专供+ 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf	
专题26: 消息队列面试题: RabbitMQ、Kafka、RocketMQ (卷王专供+ 史上最全 + 2023面试必备) -V10-from-Java面试红宝书-release.pdf	
专题27: 内存泄漏 内存溢出 (卷王专供+ 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf	
专题28: JVM 内存溢出 实战 (卷王专供+ 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf	
专题29: 多线程面试题 (卷王专供+ 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf	
专题30: HR面试题: 过五关斩六将后, 小心阴沟翻船! (史上最全、避坑宝典) -V2-from-Java面试红宝书-release.pdf	
专题31: Hash链表面试题 (卷王专供+ 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf	
专题32: 大厂面试的基本流程和面试准备 (阿里、腾讯、网易、京东、头条.....) -V2-from-Java面试红宝书-release.pdf	
专题33: BST、AVL、RBT红黑树、三大核心数据结构 (卷王专供+ 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf	
专题34: Elasticsearch面试题 (卷王专供+ 史上最全 + 2023面试必备) -V3-from-Java面试红宝书-release.pdf	
专题35: Mybatis面试题 (卷王专供+ 史上最全 + 2023面试必备) -V3-from-尼恩Java面试宝典-release.pdf	

海量定时任务管理的问题

下面的问题，来自互联网：

一个大型内容审核平时，在运营设定审核了内容的通过的时间，到了这个时间之后，相关内容自动审核通过，

本是个小的需求，但是考虑到如果需要定时审核的东西很多，这样大量的定时任务带来的一系列问题，

海量定时任务管理的场景非常多，在实际项目中，存在大量需要定时或是延时触发的任务，

比如电商中，延时需要检查订单是否支付成功，是否配送成功，定时给用户推送提醒等等

方案一 单定时器方案

描述：

把所有需要定时审核的资源放到redis中，例如sorted set中，需要审核通过的时间作为score值。

后台启动一个定时器，定时轮询sortedSet，当score值小于当前时间，则运行任务审核通过。

问题

这个方案在小批量数据的情况下没有问题，

但是在大批量任务的情况下就会出现问题了，因为每次都要轮询全量的数据，逐个判断是否需要执行，一旦轮询任务执行比较长，就会出现任务无法按照定时的时间执行的问题。

方案二 多定时器方案

描述

每个需要定时完成的任务都启动一个定时任务，然后等待完成之后销毁

问题

这个方案带来的问题很明显，定时任务比较多的情况下，会启动很多的线程，这样服务器会承受不了之后崩溃。

基本上不会采取这个方案。

方案三 redis的过期通知功能

描述

和方案一类似，针对每一个需要定时审核的任务，设定过期时间，

过期时间也就是审核通过的时间，订阅redis的过期事件，当这个事件发生时，执行相应的审核通过任务。

问题

这个方案来说是借用了redis这种中间件来实现我们的功能，这中实际上属于redis的发布订阅功能中的一部分，

针对redis发布订阅功能是不推荐我们在生产环境中做业务操作的，

通常redis内部（例如redis集群节点上下线，选举等等来使用），我们业务系统使用它的这个事件会产生如下两个问题

- 1、redis发布订阅的不稳定问题
- 2、redid发布订阅的可靠性问题

具体可以参考 <https://my.oschina.net/u/2457218/blog/3065021> (redis的发布订阅缺陷)

方案四 Hash分层记时轮（分层时间轮）算法

这个东西就是专为大批量定时任务管理而生。

具体论文详见参考文献

<http://www.cs.columbia.edu/~nahum/w6998/papers/sosp87-timing-wheels.pdf>

什么是时间轮

时间轮这个技术其实出来很久了，在kafka、zookeeper等技术中都有时间轮使用的方式。

时间轮的基本概念

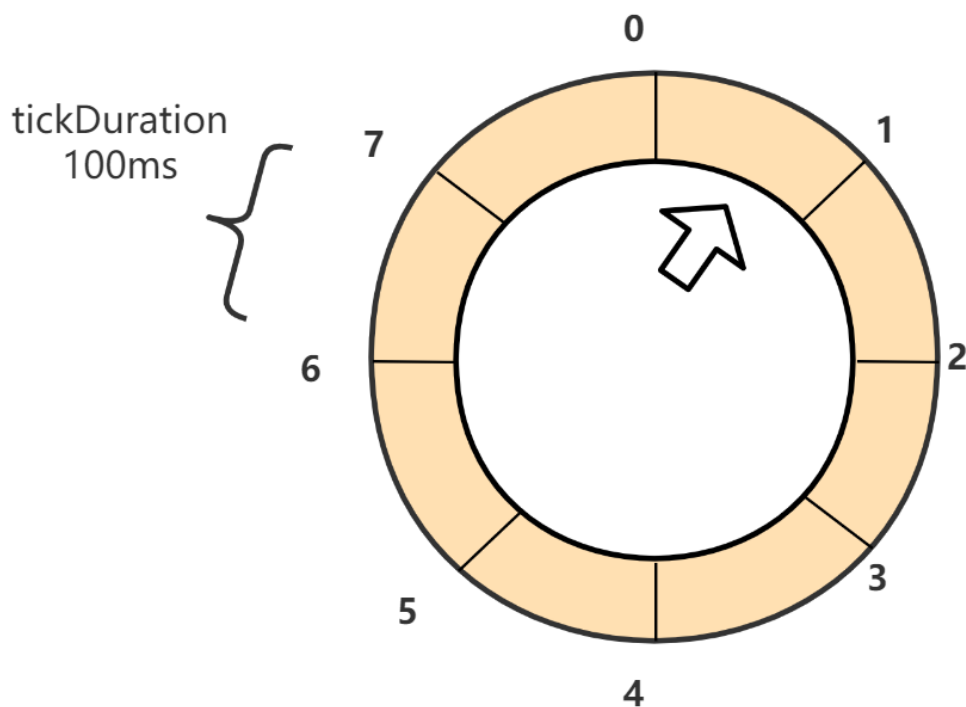
时间轮这个技术其实出来很久了，在kafka、zookeeper、Netty、Dubbo等高性能组件中都有时间轮使用的方式。

如图，时间轮，从图片上来看，就和手表的表圈是一样，所以称为时间轮。



时间轮其实就是一种环形的数据结构,其设计参考了时钟转动的思维,可以想象成时钟,分成很多格子,一个格子代表一段时间

时间轮是由多个时间格组成,下图中有8个时间格,每个时间格代表当前时间轮的基本时间跨度 (tickDuration) , 其中时间轮的时间格的个数是固定的。



CSDN @40岁资深老架构师尼恩

图中，有8个时间格（槽），假设每个时间格的单位为100ms，那么整个时间轮走完一圈需要800ms。

每100ms指针会沿着顺时针方向移动一个时间单位，

这个单位可以代表时间精度

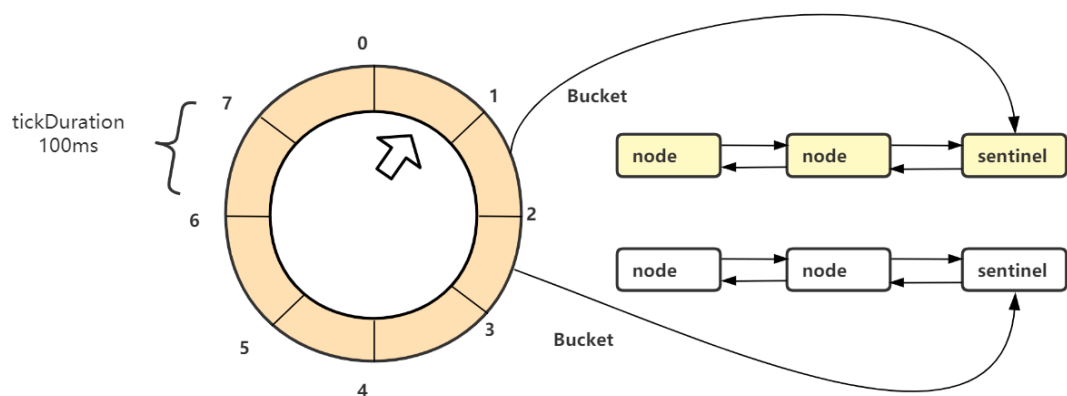
这个单位可以设置，

比如以秒为单位，也可以以一小时为单位。

通过指针移动，来获得每个时间格中的任务列表，然后遍历这一个时间格中的双向链表来执行任务，以此循环。

时间轮是以时间作为刻度，组成的一个环形队列，这个环形队列采用数组来实现，

数组的每个元素称为槽 Bucket，



CSDN @40岁资深老架构师尼恩

每个槽位可以放一个定时任务列表，叫HashedWheelBucket，

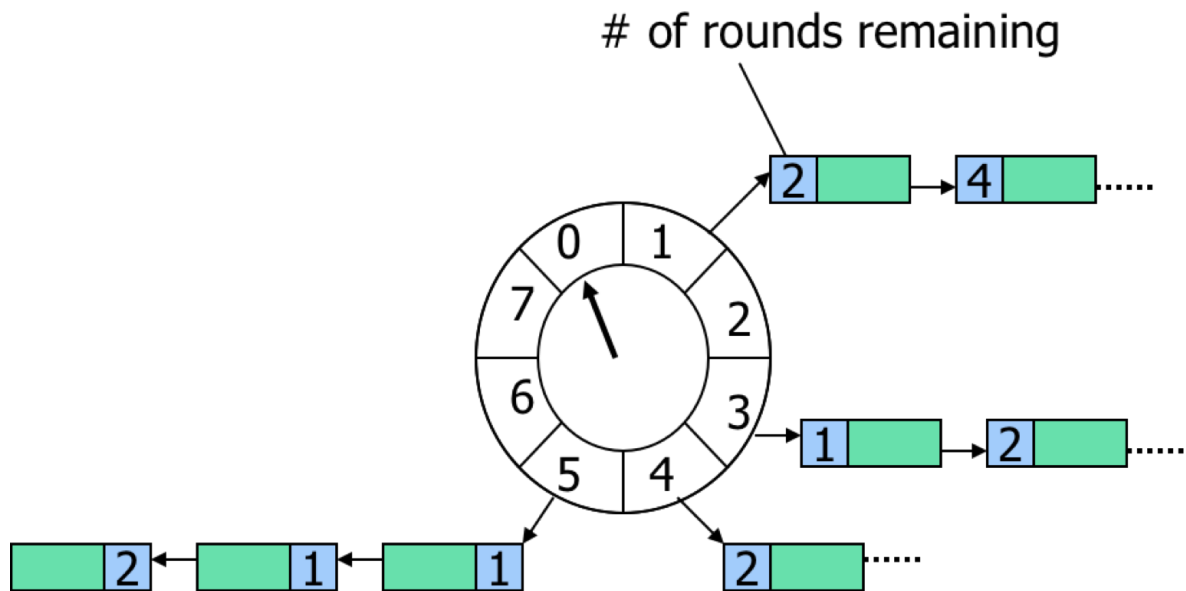
每个槽位可以是一个**双向链表**，其中可以设置一个 sentinel 哨兵节点，作为添加任务和删除任务的起始节点。

槽位链表的每一项表示一个定时任务项（HashedWheelTimeout），其中封装了真正的定时任务 TimerTask。

简单来说：

时间轮是一种高效利用线程资源进行批量化调度的一种调度模型。

把大批量的调度任务全部绑定到同一个调度器上，使用这一个调度器来进行所有任务的管理、触发、以及运行。



时间轮的模型能够高效管理各种任务：

- 延时任务、
- 周期任务、
- 通知任务。

时间轮与基于排序链表的定时器的不同：

在时间轮内，指针指向轮子上的一个槽。

它以恒定的速率顺时针转动。没转动一步就指向下一个槽，每次转动称之为一个tick。

一个滴答的时间称为时间轮的槽间隔 s_i （slot interval），它实际上就是心搏时间。

时间轮共有 N 个槽，因此它运转一周的时间是 $N \cdot s_i$ 。

每个槽指向一个定时器链表，每条链表上的定时器具有相同的特征：**它们的定时时间相差 $N \cdot s_i$ 的整数倍。**

时间轮正式利用这个关系将定时器散列到不同的链表中。

假如：现在指针指向槽 cs ，我们要添加一个定时时间为 ti 的定时器，则该定时器将被插入槽 ts （timer slot）对应的链表中：

$$ts = (cs + (ti / s_i)) \% N$$

时间轮与基于排序链表的定时器的不同：

基于排序链表的定时器，使用唯一的链表来管理所有定时器，所以插入操作的效率随着定时器数目的增多而降低。

而时间轮使用**哈希表**的思想，将定时器散列到不同的链表上。

这样每条链表上的定时器数目都将明显少于原来的排序链表上的定时器数目，插入操作的效率基本不受定时器数目的影响。

很显然，对时间轮而言：

- 要提高定时精度，就要使si值足够小，时间间隔越短，精度越高；
- 要提高执行效率，则要求N值足够大，槽位越多，每个槽位的任务越少。

上图描述的是一个简单的时间轮，仅仅一个轮子。

而复杂的时间轮可能有多个轮子，不同轮子拥有不同的粒度。

时间轮的特点

时间轮是一个高性能，低消耗的数据结构，它适合用非准实时，延迟的短平快任务，例如心跳检测。

比如Netty动辄管理100w+的连接，每一个连接都会有很多超时任务。

比如发送超时、心跳检测间隔等，如果每一个定时任务都启动一个Timer，不仅低效，而且会消耗大量的资源。

在Netty中的一个典型应用场景是判断某个连接是否idle，如果idle（如客户端由于网络原因导致到服务器的心跳无法送达），则服务器会主动断开连接，释放资源。

得益于Netty NIO的优异性能，基于Netty开发的服务器可以维持大量的长连接，单台8核16G的云主机可以同时维持几十万长连接，及时掐掉不活跃的连接就显得尤其重要。

时间轮的使用场景

时间轮的本质是一种类似延迟任务队列的实现，

那么它的特点如上所述，适用于对时效性不高的，可快速执行的，大量这样的“小”任务，能够做到高性能，低消耗。

应用场景大致有：

- 心跳检测(客户端探活)
- 会话、请求是否超时
- 消息延迟推送
- 业务场景超时取消(订单、退款单等)

时间轮的思想应用范围非常广泛，各种操作系统的定时任务调度，Crontab,还有基于java的通信框架Netty中也有时间轮的实现，

几乎所有的时间任务调度系统采用的都是时间轮的思想。

至于采用round型的基础时间轮还是采用分层时间轮，看实际需要吧，时间复杂度和实现复杂度的取舍。

时间轮的使用

这里使用的时间轮是Netty这个包中提供的，使用方法比较简单。

先构建一个HashedWheelTimer时间轮。

- tickDuration: 100，表示每个时间格代表当前时间轮的基本时间跨度，这里是100ms，也就是指针100ms跳动一次，每次跳动一个窗格
- ticksPerWheel: 1024，表示时间轮上一共有多少个窗格，分配的窗格越多，占用内存空间就越大
- leakDetection: 是否开启内存泄漏检测。
- maxPendingTimeouts[可选参数]，最大允许等待的任务数，默认没有限制。
- 通过newTimeout() 把需要延迟执行的任务添加到时间轮中

```
CountDownLatch countDownLatch = new CountDownLatch(1);

HashedWheelTimer timer = new HashedWheelTimer(1, TimeUnit.SECONDS, 16);
log.info("current timestamp={}", System.currentTimeMillis());
timer.newTimeout((timeout) -> {
    log.info("task execute,current timestamp={}", System.currentTimeMillis());
    countDownLatch.countDown();
}, 2000, TimeUnit.MILLISECONDS);

countDownLatch.await();
timer.stop();
```

时间轮的原理解析

时间轮的整体原理，分为几个部分。

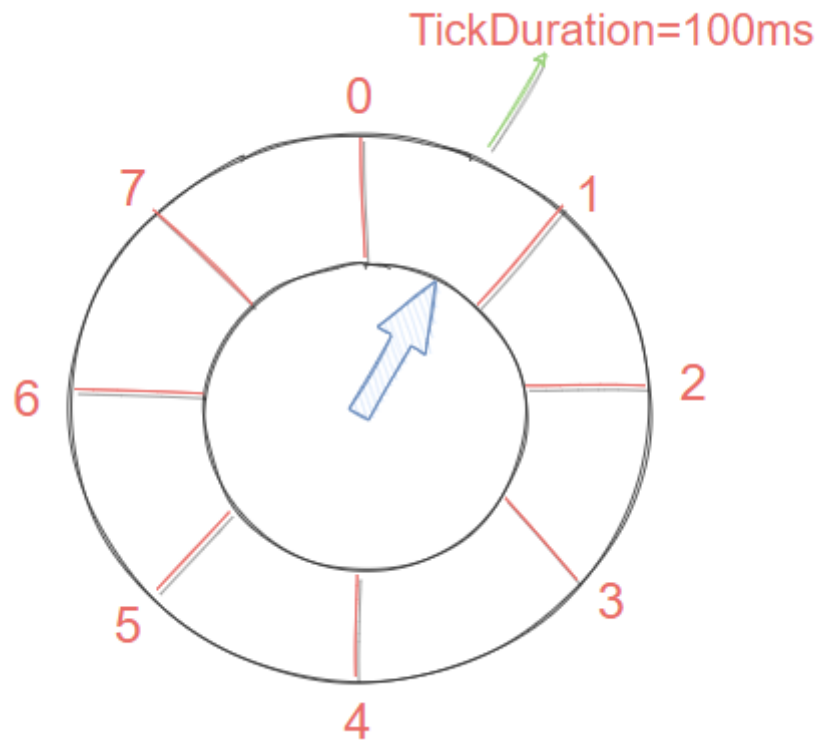
时间轮至少需要提供4个功能：

1. 加入任务
2. 执行任务
3. 删除任务
4. 沿着时间刻度前进

创建时间轮

时间轮本质上是一个环状数组，比如我们初始化时间轮时：ticksPerWheel=8，那么意味着这个环状数组的长度是8，如图3-12所示。

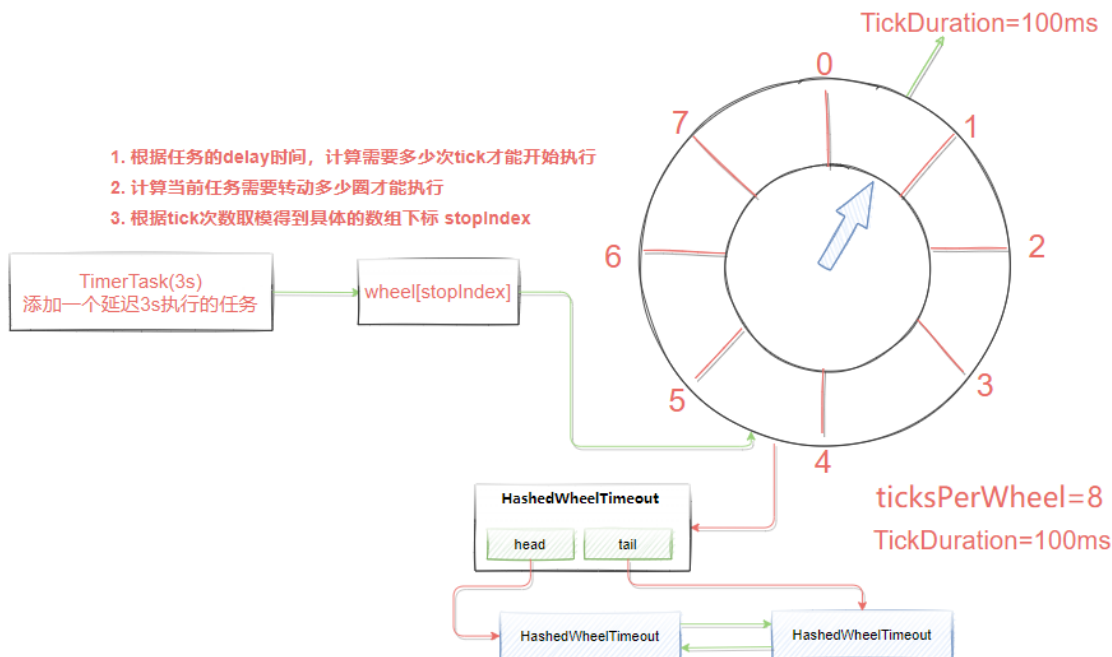
```
HashedWheelBucket[] wheel = new HashedWheelBucket[ticksPerWheel];
```



添加任务

- 当通过newTimeout()方法添加一个延迟任务时，该任务首先会加入到一个阻塞队列中。
- 然后会有一个定时任务从该队列获取任务，添加到时间轮的指定位置，计算方法如下。

```
//当前任务的开始执行时间除以每个窗口的时间间隔，得到一个calculated值（表示需要经过多少tick，  
指针没跳动一个窗格，tick会递增），单位为nanos（微毫秒）long calculated =  
timeout.deadline / tickDuration;//计算当前任务需要在时间轮中经历的圈数，因为当前任务执行  
时间有可能大于完整一圈的时间，所以需要计算经过几圈之后才能执行该任务。  
timeout.remainingRounds = (calculated - tick) / wheel.length;//取最大的一个tick，有  
可能当前任务在队列中已经过了执行时间，这种情况下直接用calculated这个值就没意义了。final long  
ticks = Math.max(calculated, tick); // Ensure we don't schedule for past.int  
stopIndex = (int) (ticks & mask); //通过ticks取模mask，得到一个下标HashedWheelBucket  
bucket = wheel[stopIndex]; //把任务添加到指定数组下标位置
```



任务执行

Worker线程按照每次间隔时间转动后，得到该时间窗格中的任务链表，然后从链表的head开始逐个取出任务，有两个判断条件

- 当前任务需要转动的圈数为0，表示任务是当前圈开始执行
- 当前任务达到了delay时间，也就是 `timeout.deadline <= deadline`
- 最终调用`timeout.expire()`方法执行任务。

```
public void expireTimeouts(long deadline) {
    HashedWheelTimeout timeout = head;
    // process all timeouts while (timeout != null) {
    HashedWheelTimeout next = timeout.next;
    if (timeout.remainingRounds <= 0) {
        next = remove(timeout);
        if (timeout.deadline <= deadline) {
            timeout.expire();
        } else {
            // The timeout was placed into a wrong slot. This should never happen.
            throw new IllegalStateException(String.format(
                "timeout.deadline (%d) > deadline (%d)", timeout.deadline, deadline));
        }
    } else if (timeout.isCancelled()) {
        next = remove(timeout);
    } else {
        timeout.remainingRounds--;
        timeout = next;
    }
}
```

同一时刻存在多个任务

更通用的情况，同一时刻可能需要执行多个任务，

比如：

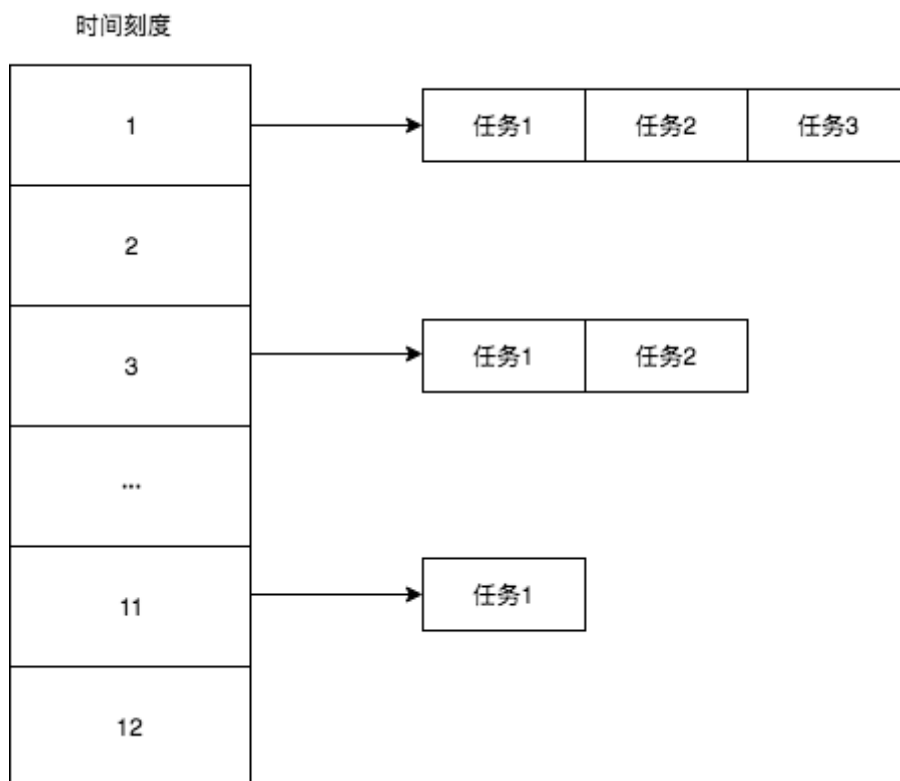
- 每天上午九点除了生成报表之外，
- 执行发送邮件的任务，
- 执行创建文件的任务，
- 执行数据分析的任务

- 等等,

为了存储这些任务，时间轮的数据结构其实类似hashmap，只是key时间刻度值，并且不做hash运算

每一个时间刻度，可以理解为一个槽位，

如果有多个任务需要执行呢？一个槽位可以指向一个数组或者链表，用来存放该刻度需要执行的任务，



同一时刻存在多个任务时，只要把该刻度对应的链表全部遍历一遍，执行（扔到线程池中异步执行）其中的任务即可。

时间刻度不够用怎么办？

如果任务不只限定在一天之内呢？

比如我有个任务，需要每周一上午九点执行，我还有另一个任务，需要每周三上午九点执行。

大概的解决办法是：

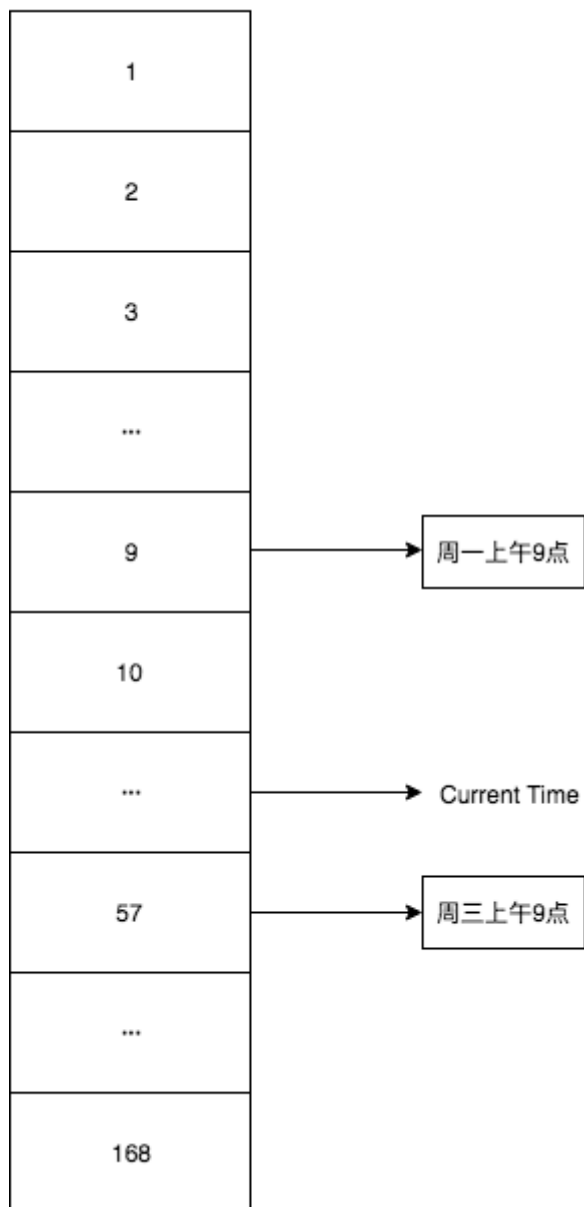
- 增大时间轮的刻度
- 列表中的任务中添加round属性
- 分层时间轮

增大时间轮的刻度

一天24个小时，一周168个小时，为了解决上面的问题，我可以把时间轮的刻度（槽）从12个增加到168个，

比如现在是星期二上午10点钟，那么下周一上午九点就是时间轮的第9个刻度，这周三上午九点就是时间轮的第57个刻度，

示意图如下：
时间刻度



仔细思考一下，会发现这中方式存在几个缺陷：

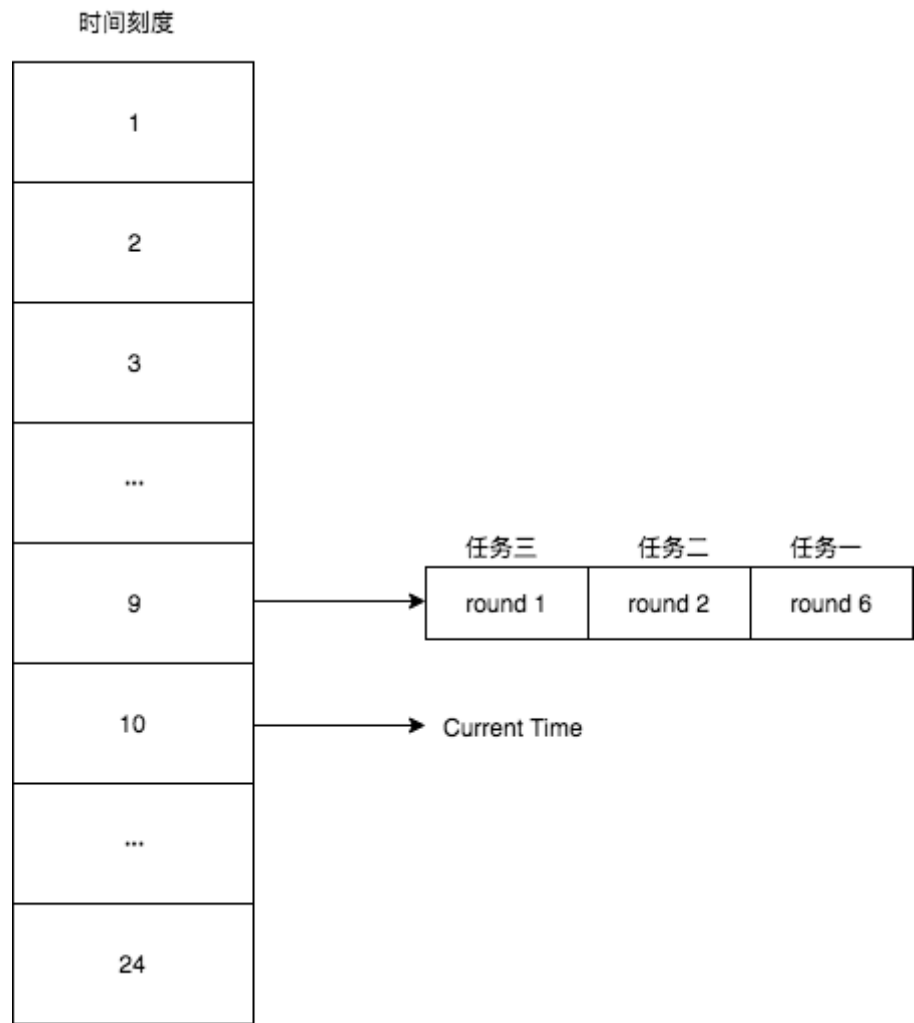
1. 时间刻度太多会导致时间轮走到的多数刻度没有任务执行，比如一个月就2个任务，我得移动720次，其中718次是无用功。
 2. 时间刻度太多会导致存储空间变大，利用率变低，比如一个月就2个任务，我得需要大小是720的数组，如果我的执行时间的粒度精确到秒，那就更恐怖了。
- 于是乎，聪明的你脑袋一转，想到另一个办法：

列表中的任务中添加round属性

这次我不增加时间轮的刻度了，刻度还是24个，现在有三个任务需要执行，

1. 任务一每周二上午九点。
2. 任务二每周四上午九点。
3. 任务三每个月12号上午九点。

比如现在是9月11号星期二上午10点，时间轮转一圈是24小时，到任务一下次执行（下周二上午九点），
需要时间轮转过6圈后，到第7圈的第9个刻度开始执行。
任务二下次执行第3圈的第9个刻度，任务三是第2圈的第9个刻度。
示意图如下：



时间轮每移动到一个刻度时，遍历任务列表，把round值-1，然后取出所有round=0的任务执行。

这样做能解决时间轮刻度范围过大造成的空间浪费，但是却带来了另一个问题：

- 时间轮每次都需要遍历任务列表，耗时增加，当时间轮刻度粒度很小(秒级甚至毫秒级)，
- 任务列表又特别长时，这种遍历的办法是不可接受的。

当然，对于大多数场景，这种方法还是适用的。

有没有既节省空间，又节省时间的办法呢？

答案是有的，正如《Hashed and Hierarchical Timing Wheels》标题中提到的，有一种分层时间轮，可以解决做到既节省空间，又节省时间：

分层时间轮

分层时间轮是这样一种思想：

1. 针对时间复杂度的问题：

不做遍历计算round，凡是任务列表中的都应该是应该被执行的，直接全部取出来执行。

2. 针对空间复杂度的问题：

分层，每个时间粒度对应一个时间轮，多个时间轮之间进行级联协作。

第一点很好理解，第二点有必要举个例子来说明。

比如我有三个任务：

- 任务一每周二上午九点。
- 任务二每周四上午九点。
- 任务三每个月12号上午九点。

三个任务涉及到四个时间单位：小时、天、星期、月份。

拿任务三来说，任务三得到执行的前提是，

时间刻度先得来到12号这一天，然后才需要关注其更细一级的时间单位：上午9点。

基于这个思想，我们可以设置三个时间轮：月轮、周轮、天轮。

- 月轮的时间刻度是天。
- 周轮的时间刻度是天。
- 天轮的时间刻度是小时。

初始添加任务时：

- 任务一添加到周轮上，
- 任务二添加到周轮上
- 任务三添加到月轮上。

三个时间轮以各自的时间刻度不停流转。

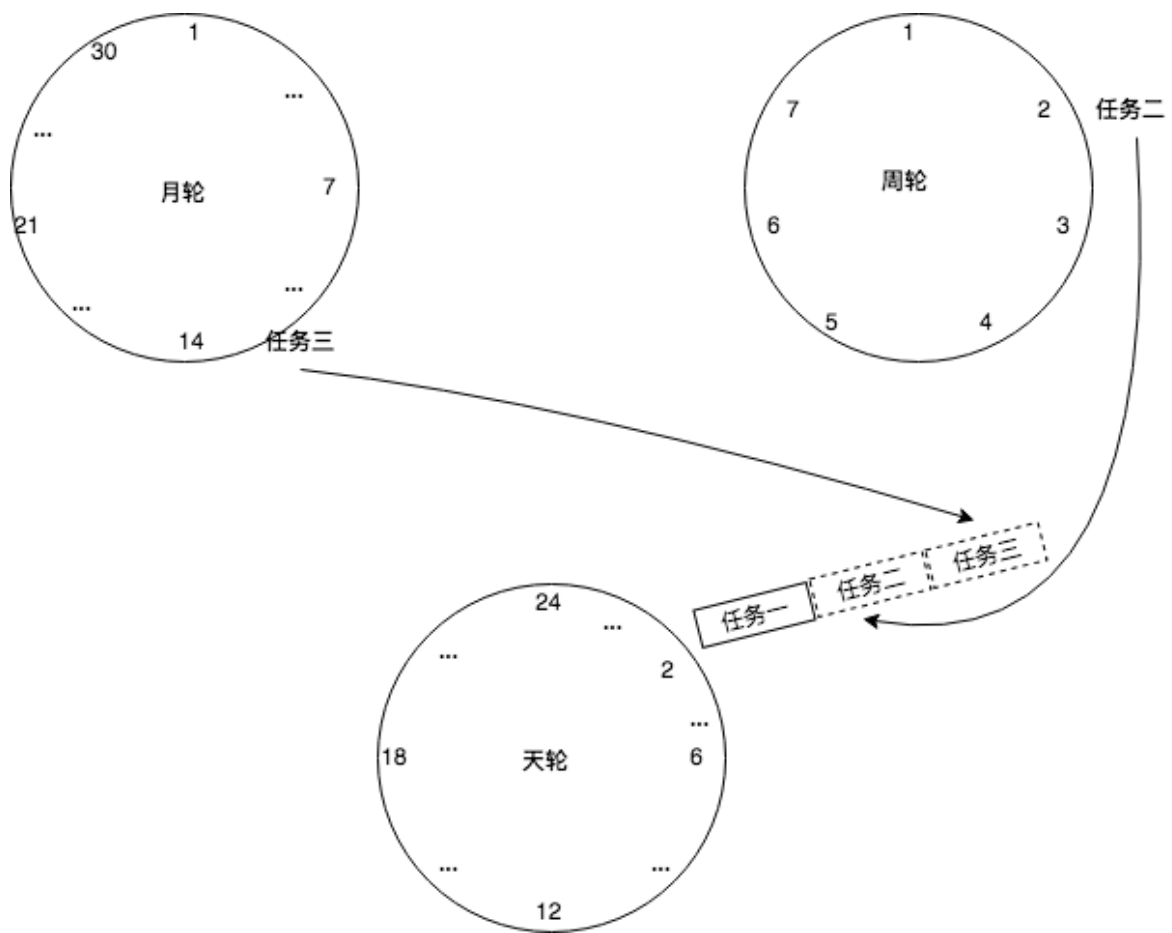
当周轮移动到刻度2(星期二)时，取出这个刻度下的任务1，丢到天轮上，天轮接管该任务，到9点执行。

当周轮移动到刻度4(周四)时，取出这个刻度下的任务2，丢到天轮上，天轮接管该任务，到9点执行。

当月轮移动到刻度12(12号)时，取出这个刻度下的任务，丢到天轮上，天轮接管该任务，到9点执行。

这样就可以做到既不浪费空间，有不浪费时间。

整体的示意图如下所示：



时间轮的源码分析

HashedWheelTimer的构造

- 调用createWheel创建一个时间轮，时间轮数组一定是2的幂次方，比如传入的ticksPerWheel=6，那么初始化的wheel长度一定是8，这样是便于时间格的计算。
- tickDuration，表示时间轮的跨度，代表每个时间格的时间精度，以纳秒的方式来表现。
- 把工作线程Worker封装成WorkerThread，从名字可以知道，它就是最终那个负责干活的线程。

```

public HashedWheelTimer(    ThreadFactory threadFactory,    long tickDuration,
TimeUnit unit, int ticksPerWheel,    long maxPendingTimeouts) {    // 创建时间轮基
本的数据结构，一个数组。长度为不小于ticksPerWheel的最小2的n次方    wheel =
createWheel(ticksPerWheel);    // 这是一个标示符，用来快速计算任务应该呆的格子。    //
我们知道，给定一个deadline的定时任务，其应该呆的格子=deadline%wheel.length.但是%操作是个相
对耗时的操作，所以使用一种变通的位运算代替：    // 因为一圈的长度为2的n次方，mask = 2^n-1后
低位将全部是1，然后deadline&mask == deadline%wheel.length    // java中的HashMap在进行
hash之后，进行index的hash寻址寻址的算法也是和这个一样的    mask = wheel.length - 1;
//时间轮的基本时间跨度，（tickDuration传入是1的话，这里会转换成1000000）
this.tickDuration = unit.toNanos(tickDuration);    // 校验是否存在溢出。即指针转动的
时间间隔不能太长而导致tickDuration*wheel.length>Long.MAX_VALUE    if
(this.tickDuration >= Long.MAX_VALUE / wheel.length) {        throw new
IllegalArgumentException(String.format(            "tickDuration: %d (expected:
0 < tickDuration in nanos < %d",                tickDuration, Long.MAX_VALUE /
wheel.length));    }    //把worker包装成thread    workerThread =
threadFactory.newThread(worker);    this.maxPendingTimeouts =
maxPendingTimeouts;    //如果HashedWheelTimer实例太多，那么就会打印一个error日志    if
(INSTANCE_COUNTER.incrementAndGet() > INSTANCE_COUNT_LIMIT &&
WARNED_TOO_MANY_INSTANCES.compareAndSet(false, true)) {
reportTooManyInstances();    }}

```

- 对传入的ticksPerWheel进行整形
- 初始化固定长度的HashedWheelBucket

```

private static HashedWheelBucket[] createWheel(int ticksPerWheel) {    if
(ticksPerWheel <= 0) {        throw new IllegalArgumentException(
"ticksPerWheel must be greater than 0: " + ticksPerWheel);    }    if
(ticksPerWheel > 1073741824) {        throw new IllegalArgumentException(
"ticksPerWheel may not be greater than 2^30: " + ticksPerWheel);    }    //
对传入的时间轮大小进行整形，整形成2的幂次方    ticksPerWheel =
normalizeTicksPerWheel(ticksPerWheel);    //初始化一个固定长度的Bucket数组
HashedWheelBucket[] wheel = new HashedWheelBucket[ticksPerWheel];    for (int i
= 0; i < wheel.length; i++) {        wheel[i] = new HashedWheelBucket();    }
return wheel;}

```

添加任务到时间轮

完成时间轮的初始化之后，并没有去启动时间轮，继续看FailbackClusterInvoker中的代码。

构建了一个RetryTimerTask，也就是一个重试的定时任务，接着把这个任务通过newTimeout加入到时间轮中，其中

- retryTimerTask，表示具体的重试任务
- RETRY_FAILED_PERIOD，表示重试间隔时间，默认为5s

```
RetryTimerTask retryTimerTask = new RetryTimerTask(loadbalance, invocation,
invokers, lastInvoker, retries,
RETRY_FAILED_PERIOD);failTimer.newTimeout(retryTimerTask, RETRY_FAILED_PERIOD,
TimeUnit.SECONDS);
```

调用newTimeout方法，把任务添加进来。

```
public Timeout newTimeout(TimerTask task, long delay, TimeUnit unit) {    if
(task == null) {        throw new NullPointerException("task");    }    if (unit
== null) {        throw new NullPointerException("unit");    }    //统计任务个数
long pendingTimeoutsCount = pendingTimeouts.incrementAndGet();    //判断最大任务数
量是否超过限制    if (maxPendingTimeouts > 0 && pendingTimeoutsCount >
maxPendingTimeouts) {        pendingTimeouts.decrementAndGet();        throw new
RejectedExecutionException("Number of pending timeouts ("
+ pendingTimeoutsCount + ") is greater than or equal to
maximum allowed pending "
+ "timeouts (" + maxPendingTimeouts + ")");    }    //如果时间轮没有启动，则通过start方
法进行启动    start();    // Add the timeout to the timeout queue which will be
processed on the next tick.    // During processing all the queued
HashedWheelTimeouts will be added to the correct HashedWheelBucket.    //计算任务
的延迟时间，通过当前的时间+当前任务执行的延迟时间-时间轮启动的时间。    long deadline =
System.nanoTime() + unit.toNanos(delay) - startTime;    //在delay为正数的情况下，
deadline是不可能为负数    //如果为负数，那么说明超过了long的最大值    if (delay > 0 &&
deadline < 0) {        deadline = Long.MAX_VALUE;    }    //创建一个Timeout任务，理
论上来说，这个任务应该要加入到时间轮的时间格子中，但是这里并不是先添加到时间格，而是先    //加
入到一个阻塞队列，然后等到时间轮执行到下一个格子时，再从队列中取出最多100000个任务添加到指定的
时间格（槽）中。    HashedWheelTimeout timeout = new HashedWheelTimeout(this, task,
deadline);    timeouts.add(timeout);    return timeout;}
```

start

任务添加到阻塞队列之后，我们再来看启动方法

start方法会根据当前的workerState状态来启动时间轮。并且用了startTimeInitialized来控制线程的运行，如果workerThread没有启动起来，那么newTimeout方法会一直阻塞在运行start方法中。如果不阻塞，newTimeout方法会获取不到startTime。

```
public void start() {    //workerState一开始的时候是0 (WORKER_STATE_INIT)，然后才会
    设置为1 (WORKER_STATE_STARTED)    switch (WORKER_STATE_UPDATER.get(this)) {
    case WORKER_STATE_INIT:                if (WORKER_STATE_UPDATER.compareAndSet(this,
    WORKER_STATE_INIT, WORKER_STATE_STARTED)) {                workerThread.start();
        }                break;                case WORKER_STATE_STARTED:
    break;                case WORKER_STATE_SHUTDOWN:                throw new
    IllegalStateException("cannot be started once stopped");                default:
        throw new Error("Invalid WorkerState");    }    // 等待worker线程初始化时间轮的启
    动时间    while (startTime == 0) {        try {                //这里使用
    countDownLatch来确保调度的线程已经被启动                startTimeInitialized.await();
        } catch (InterruptedException ignore) {                // Ignore - it will be
    ready very soon.        }    } }
```

启动时间轮

调用start () 方法，会调用workerThread.start();来启动一个工作线程，这个工作线程是在构造方法中初始化的，包装的是一个Worker内部线程类。

所以直接进入Worker这个类的run方法，了解下它的设计逻辑

```

public void run() {    // 初始化startTime, 表示时间轮的启动时间    startTime =
System.nanoTime();    if (startTime == 0) {        // We use 0 as an indicator
for the uninitialized value here, so make sure it's not 0 when initialized.
        startTime = 1;    }    // 唤醒被阻塞的start()方法。
startTimeInitialized.countDown();    do {        //返回每tick一次的时间间隔
final long deadline = waitForNextTick();        if (deadline > 0) {
//计算时间轮的槽位        int idx = (int) (tick & mask);        //移除掉
CancelledTask        processCancelledTasks();        //得到当前指针位置的时
间槽        HashedWheelBucket bucket =        wheel[idx];
//将newTimeout()方法中加入到待处理定时任务队列中的任务加入到指定的格子中
transferTimeoutsToBuckets();        //运行目前指针指向的槽中的bucket链表中的任务
        bucket.expireTimeouts(deadline);        tick++;    } } while
(WORKER_STATE_UPDATER.get(HashedWheelTimer.this) == WORKER_STATE_STARTED);
//如果Worker_State一直是started状态, 就一直循环    // Fill the unprocessedTimeouts
so we can return them from stop() method.    for (HashedWheelBucket bucket :
wheel) {        bucket.clearTimeouts(unprocessedTimeouts); //清除时间轮中不需要处理
的任务    }    for (; ; ) {        //遍历任务队列, 发现如果有任务被取消, 则添加到
unprocessedTimeouts,也就是不需要处理的队列中。        HashedWheelTimeout timeout =
timeouts.poll();        if (timeout == null) {            break;        }
if (!timeout.isCancelled()) {            unprocessedTimeouts.add(timeout);
        }    }    //处理被取消的任务.    processCancelledTasks();}

```

时间轮指针跳动

这个方法的主要作用就是返回下一个指针指向的时间间隔, 然后进行sleep操作。

大家可以想象一下, 一个钟表上秒与秒之间是有时间间隔的, 那么waitForNextTick就是根据当前时间计算出跳动到下个时间的时间间隔, 然后进行sleep, 然后再返回当前时间距离时间轮启动时间的时间间隔。

说得再直白一点: , 假设当前的tickDuration的间隔是1s, tick默认=0, 此时第一次进来, 得到的deadline=1, 也就是下一次跳动的时间间隔是1s。假设当前处于

```

private long waitNextTick() {    //tick表示总的tick数    //tickDuration表示每个
时间格的跨度，所以deadline返回的是下一次时间轮指针跳动的时间    long deadline =
tickDuration * (tick + 1);    for (; ; ) {        //计算当前时间距离启动时间的时间间隔
        final long currentTime = System.nanoTime() - startTime;        //通过下一次
指针跳动的延迟时间距离当前时间的差额，这个作为sleep时间使用。        // 其实线程是以睡眠一定
的时候再来执行下一个ticket的任务的        long sleepTimeMs = (deadline - currentTime
+ 999999) / 1000000;        //sleepTimeMs小于零表示走到了下一个时间槽位置        if
(sleepTimeMs <= 0) {            if (currentTime == Long.MIN_VALUE) {
                return -Long.MAX_VALUE;            } else {                return
currentTime;            }        }        if (isWindows()) {
sleepTimeMs = sleepTimeMs / 10 * 10;        }        //进入到这里进行sleep，表示当前
时间距离下一次tick时间还有一段距离，需要sleep。        try {
Thread.sleep(sleepTimeMs);        } catch (InterruptedException ignored) {
        if (WORKER_STATE_UPDATER.get(HashedWheelTimer.this) ==
WORKER_STATE_SHUTDOWN) {            return Long.MIN_VALUE;        }
    }    }
}

```

transferTimeoutsToBuckets

转移任务到时间轮中，前面我们讲过，任务添加进来时，是先放入到阻塞队列。

而在现在这个方法中，就是把阻塞队列中的数据转移到时间轮的指定位置。

在这个转移方法中，写死了一个循环，每次都只转移10万个任务。

然后根据HashedWheelTimeout的deadline延迟时间计算出时间轮需要运行多少次才能运行当前的任务，如果当前的任务延迟时间大于时间轮跑一圈所需要的时间，那么就计算需要跑几圈才能到这个任务运行。

最后计算出该任务在时间轮中的槽位，添加到时间轮的链表中。

```
private void transferTimeoutsToBuckets() { // 循环100000次，也就是每次转移10w个任务
    for (int i = 0; i < 100000; i++) { //从阻塞队列中获得具体的任务
        HashedWheelTimeout timeout = timeouts.poll(); if (timeout == null) {
            // all processed break; } if (timeout.state() ==
            HashedWheelTimeout.ST_CANCELLED) { // Was cancelled in the meantime.
                continue; } //计算tick次数，deadline表示当前任务的延迟时间，
        tickDuration表示时间槽的间隔，两者相除就可以计算当前任务需要tick几次才能被执行 long
        calculated = timeout.deadline / tickDuration; // 计算剩余的轮数，只有 timer
        走够轮数，并且到达了 task 所在的 slot，task 才会过期。(被执行)
        timeout.remainingRounds = (calculated - tick) / wheel.length; //如果任务在
        timeouts队列里面放久了，以至于已经过了执行时间，这个时候就使用当前tick，也就是放到当前
        bucket，此方法调用完后就会被执行 final long ticks = Math.max(calculated,
        tick); // 算出任务应该插入的 wheel 的 slot，stopIndex = tick 次数 & mask,
        mask = wheel.length - 1 int stopIndex = (int) (ticks & mask); //把
        timeout任务插入到指定的bucket链中。 HashedWheelBucket bucket =
        wheel[stopIndex]; bucket.addTimeout(timeout); }}
```

运行时间轮中的任务

当指针跳动到某一个时间槽中时，会就触发这个槽中的任务的执行。该功能是通过expireTimeouts来实现

这个方法的主要作用是：过期并执行格子中到期的任务。也就是当tick进入到指定格子时，worker线程会调用这个方法

HashedWheelBucket是一个链表，所以我们需要从head节点往下进行遍历。如果链表没有遍历到链表尾部那么就继续往下遍历。

获取的timeout节点节点，如果剩余轮数remainingRounds大于0，那么就说明要到下一圈才能运行，所以将剩余轮数减一；

如果当前剩余轮数小于等于零了，那么就将当前节点从bucket链表中移除，并判断一下当前的时间是否大于timeout的延迟时间，如果是则调用timeout的expire执行任务。

```

void expireTimeouts(long deadline) {    HashedWheelTimeout timeout = head;    //
遍历当前时间槽中的所有任务    while (timeout != null) {        HashedWheelTimeout
next = timeout.next;        //如果当前任务要被执行，那么remainingRounds应该小于或者等于
0        if (timeout.remainingRounds <= 0) {            //从bucket链表中移除当前
timeout，并返回链表中下一个timeout            next = remove(timeout);        //
如果timeout的时间小于当前的时间，那么就调用expire执行task            if
(timeout.deadline <= deadline) {                timeout.expire();            }
else {                //不可能发生的情况，就是说round已经为0了，deadline却>当前槽的
deadline                // The timeout was placed into a wrong slot. This should
never happen.                throw new IllegalStateException(String.format(
                "timeout.deadline (%d) > deadline (%d)", timeout.deadline,
deadline));            }        } else if (timeout.isCancelled()) {
next = remove(timeout);        } else {            //因为当前的槽位已经过了，说明已经
走了一圈了，把轮数减一            timeout.remainingRounds--;        }        //把指针
放置到下一个timeout        timeout = next;    }}

```

1.1 HashedWheelTimeout方法

HashedWheelTimeout 是 Timeout 接口的唯一实现，是 HashedWheelTimer 的内部类。

HashedWheelTimeout 扮演了两个角色：

- 第一个，时间轮中双向链表的**节点**，即定时任务 TimerTask 在 HashedWheelTimer 中的容器。
- 第二个，定时任务 TimerTask 提交到 HashedWheelTimer 之后返回的**句柄** (Handle)，用于在时间轮外部查看和控制定时任务。

的核心字段如下：

- prev、next (HashedWheelTimeout类型)

分别对应当前定时任务在链表中的前驱节点和后继节点。

- task (TimerTask类型)

指实际被调度的任务。

- deadline (long类型)

指定定时任务执行的时间。这个时间是在创建 HashedWheelTimeout 时指定的，计算公式是：

currentTime (创建 HashedWheelTimeout 的时间) + delay (任务延迟时间) -

startTime (HashedWheelTimer 的启动时间)，时间单位为纳秒。

- state (volatile int类型)

指定定时任务当前所处状态，可选的有三个，分别是 INIT (0)、CANCELLED (1) 和 EXPIRED (2)。

另外，还有一个 STATE_UPDATER 字段 (AtomicIntegerFieldUpdater类型) 实现 state 状态变更的原子性。

- remainingRounds (long类型)

指当前任务剩余的时钟周期数。时间轮所能表示的时间长度是有限的，在任务到期时间与当前时刻的时间差，超过时间轮单圈能表示的时长，就出现了套圈的情况，需要该字段值表示剩余的时钟周期。

核心方法有：

- isCancelled()、isExpired()、state() 方法,

主要用于检查当前 HashedWheelTimeout 状态。

- cancel() 方法,

将当前 HashedWheelTimeout 的状态设置为 CANCELLED, 并将当前 HashedWheelTimeout 添加到 cancelledTimeouts 队列中等待销毁。

- expire() 方法,

当任务到期时, 会调用该方法将当前 HashedWheelTimeout 设置为 EXPIRED 状态, 然后调用其中的 TimerTask 的 run() 方法执行定时任务。

- remove() 方法,

将当前 HashedWheelTimeout 从时间轮中删除。

1.2 HashedWheelBucket方法

HashedWheelBucket是时间轮中的一个槽, 时间轮中的槽实际上就是一个用于缓存和管理双向链表的容器, 双向链表中的每一个节点就是一个 HashedWheelTimeout 对象, 也就关联了一个 TimerTask 定时任务。

HashedWheelBucket 持有双向链表的首尾两个节点, 分别是 head 和 tail 两个字段, 再加上每个 HashedWheelTimeout 节点均持有前驱和后继的引用, 这样就可以正向或是逆向遍历整个双向链表了。

重要属性与方法

- addTimeout() 方法:

新增 HashedWheelTimeout 到双向链表的尾部。

- remove() 方法:

从双向链表中移除指定的 HashedWheelTimeout 节点。

- clearTimeouts() 方法:

循环调用 pollTimeout() 方法处理整个双向链表, 并返回所有未超时或者未被取消的任务。

- expireTimeouts() 方法:

遍历双向链表中的全部 HashedWheelTimeout 节点。

在处理到期的定时任务时, 会通过 remove() 方法取出, 并调用其 expire() 方法执行;

对于已取消的任务, 通过 remove() 方法取出后直接丢弃;

对于未到期的任务, 会将 remainingRounds 字段 (剩余时钟周期数) 减一。

1.3 HashedWheelTimer源码

HashedWheelTimer 是 Timer 接口的实现, 它通过时间轮算法实现了一个定时器。

HashedWheelTimer 会根据当前时间轮指针选定对应的槽 (HashedWheelBucket), 从双向链表的头部开始迭代, 对每个定时任务 (HashedWheelTimeout) 进行计算, 属于当前时钟周期则取出运行, 不属于则将其剩余的时钟周期数减一操作。

重要属性与方法

- `workerState` (`volatile int`类型) :

时间轮当前所处状态, 可选值有 `init`、`started`、`shutdown`。

同时, 有相应的 `AtomicIntegerFieldUpdater` 实现 `workerState` 的原子修改。

- `startTime` (`long`类型) :

当前时间轮的启动时间, 提交到该时间轮的定时任务的 `deadline` 字段值均以该时间戳为起点进行计算。

- `wheel` (`HashedWheelBucket[]`类型) :

该数组就是时间轮的环形队列, 每一个元素都是一个槽。当指定时间轮槽数为 `n` 时, 实际上会取大于且最靠近 `n` 的 2 的幂次方值。

- `timeouts`、`cancelledTimeouts` (`LinkedBlockingQueue`类型) :

`timeouts` 队列用于缓冲外部提交时间轮中的定时任务, `cancelledTimeouts` 队列用于暂存取消的定时任务。`HashedWheelTimer` 会在处理 `HashedWheelBucket` 的双向链表之前, 先处理这两个队列中的数据。

- `tick` (`long`类型) :

该字段在 `HashedWheelTimer$Worker` 中, 是时间轮的指针, 是一个步长为 1 的单调递增计数器。

- `mask` (`int`类型) :

掩码, `mask = wheel.length - 1`, 执行 `ticks & mask` 便能定位到对应的时钟槽。

- `ticksDuration` (`long`类型) :

时间指针每次加 1 所代表的实际时间, 单位为纳秒。

- `pendingTimeouts` (`AtomicLong`类型) :

当前时间轮剩余的定时任务总数。

- `workerThread` (`Thread`类型) :

时间轮内部真正执行定时任务的线程。

- `worker` (`Worker`类型) :

真正执行定时任务的逻辑封装这个 `Runnable` 对象中。

时间轮对外提供了一个 `newTimeout()` 接口用于提交定时任务, 在定时任务进入到 `timeouts` 队列之前会先调用 `start()` 方法启动时间轮, 其中会完成下面两个关键步骤:

1. 确定时间轮的 `startTime` 字段;
2. 启动 `workerThread` 线程, 开始执行 `worker` 任务。

之后根据 `startTime` 计算该定时任务的 `deadline` 字段, 最后才能将定时任务封装成 `HashedWheelTimeout` 并添加到 `timeouts` 队列。

下面我们来分析时间轮指针一次转动的全流程。

1. 时间轮指针转动，时间轮周期开始。
2. 清理用户主动取消的定时任务，这些定时任务在用户取消时，会记录到 cancelledTimeouts 队列中。在每次指针转动的时候，时间轮都会清理该队列。
3. 将缓存在 timeouts 队列中的定时任务转移到时间轮中对应的槽中。
4. 根据当前指针定位对应槽，处理该槽位的双向链表中的定时任务。
5. 检测时间轮的状态。如果时间轮处于运行状态，则循环执行上述步骤，不断执行定时任务。如果时间轮处于停止状态，则执行下面的步骤获取到未被执行的定时任务并加入 unprocessedTimeouts 队列：遍历时间轮中每个槽位，并调用 clearTimeouts() 方法；对 timeouts 队列中未被加入槽中循环调用 poll()。
6. 最后再次清理 cancelledTimeouts 队列中用户主动取消的定时任务。

时间轮在 Netty (4.x) 中的实现方式：

一个Hash Wheel Timer是一个环形结构，可以想象成时钟，分为很多格子，

一个格子代表一段时间（越短Timer精度越高），并用一个List保存在该格子上到期的所有任务，同时一个指针随着时间流逝一格一格转动，并执行对应List中所有到期的任务。

任务通过取模(实际是通过位运算)决定应该放入哪个格子。

假设一个格子是1秒，则整个wheel能表示的时间段为8s，假如当前指针指向2，此时需要调度一个3s后执行的任务，显然应该加入到(2+3=5)的方格中，指针再走3次就可以执行了；

如果任务要在10s后执行，应该等指针走完一个round，再走2格再执行，因此应放入4，同时将任务的round属性设置为1。

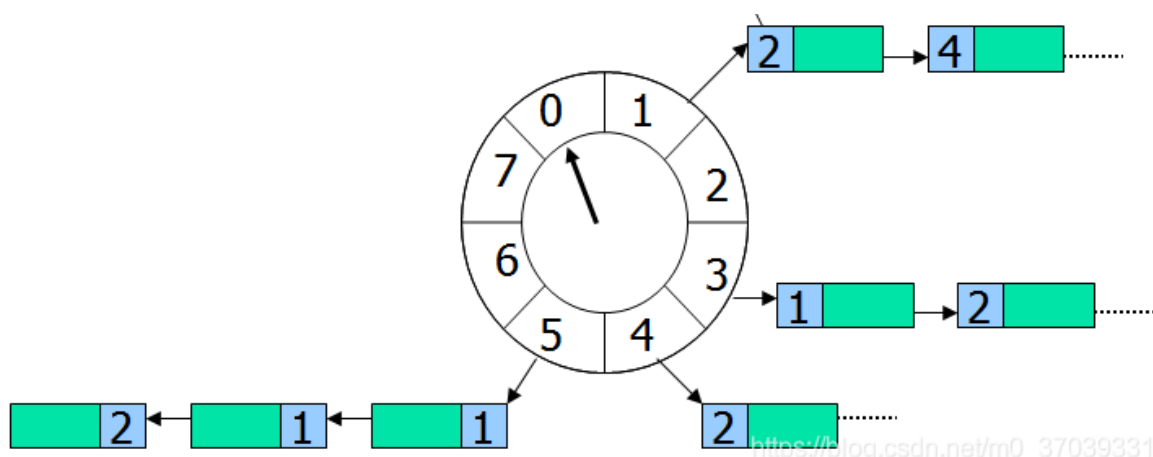
检查到期任务时应当只执行round属性为0的，格子上其他任务的round属性应减1。

schedule: $O(1)$

cancel : $O(1)$

expire : 最坏情况 $O(n)$ ，平均 $O(1)$

// 显然格子越多每个格子对应的List就越短，越接近 $O(1)$ ；最坏情况下所有的任务都在一个格子中， $O(n)$ 。



重要属性

```

//内部负责添加任务，累加tick，执行任务等。
private final worker worker = new Worker();
//负责创建worker线程。
private final Thread workerThread;

//时间刻度之间的时长(默认100ms)，通俗的说，就是多久tick++一次。
//每 tick 一次的时间间隔，每 tick 一次就会到达下一个槽位
private final long tickDuration;

//轮中的 slot 数
int ticksPerWheel;

//wheel数组元素，负责存放HashedWheelTimeout链表。
private final HashedWheelBucket[] wheel;

```

构造方法

```

//轮(Round) : 一轮的时长为 tickDuration * ticksPerWheel，也就是转一圈的时长。
public HashedWheelTimer(
    ThreadFactory threadFactory,
    long tickDuration, TimeUnit unit, int ticksPerWheel, boolean
leakDetection,
    long maxPendingTimeouts) {

    if (threadFactory == null) {
        throw new NullPointerException("threadFactory");
    }
    if (unit == null) {
        throw new NullPointerException("unit");
    }
    if (tickDuration <= 0) {
        throw new IllegalArgumentException("tickDuration must be greater than 0:
" + tickDuration);
    }
    if (ticksPerWheel <= 0) {
        throw new IllegalArgumentException("ticksPerWheel must be greater than
0: " + ticksPerWheel);
    }

    // Normalize ticksPerWheel to power of two and initialize the wheel.
    创建wheel数组，和HashMap的entry数组长度类似，为2的幂
    wheel = createWheel(ticksPerWheel);

    //用于计算任务存放wheel的索引
    //因为wheel长度为2的次方，则，如果长度为16(10000)，mask就为15(1111)
    //那么，通过    n & mask 就可以实现 类似于 n % mask，而 & 更高效.....
    mask = wheel.length - 1;

    // Convert tickDuration to nanos.

```

```

        tickDuration 不能大于 Long.MAX_VALUE / wheel.length, 也就是一轮的时间不能大于
        Long.MAX_VALUE 纳秒
        long duration = unit.toNanos(tickDuration);

        // Prevent overflow.
        // 校验是否存在溢出。
        // 即指针转动的时间间隔不能太长。
        // 太长导致tickDuration*wheel.length>Long.MAX_VALUE没法计算了
        if (duration >= Long.MAX_VALUE / wheel.length) {
            throw new IllegalArgumentException(String.format(
                "tickDuration: %d (expected: 0 < tickDuration in nanos < %d",
                tickDuration, Long.MAX_VALUE / wheel.length));
        }

        if (duration < MILLISECOND_NANOS) {
            if (logger.isWarnEnabled()) {
                logger.warn("Configured tickDuration %d smaller then %d, using
1ms.",
                                tickDuration, MILLISECOND_NANOS);
            }
            this.tickDuration = MILLISECOND_NANOS;
        } else {
            this.tickDuration = duration;
        }

        //创建worker线程
        workerThread = threadFactory.newThread(worker);

        // 这里默认是启动内存泄露检测: 当HashedWheelTimer实例超过当前cpu可用核数*4的时候, 将发出警告
        leak = leakDetection || !workerThread.isDaemon() ? leakDetector.track(this)
: null;

        //最长的执行任务的时间
        this.maxPendingTimeouts = maxPendingTimeouts;

        //检测时间轮是否太多
        if (INSTANCE_COUNTER.incrementAndGet() > INSTANCE_COUNT_LIMIT &&
            WARNED_TOO_MANY_INSTANCES.compareAndSet(false, true)) {
            reportTooManyInstances();
        }
    }
}

```

看看启动的时候都做了些什么:

启动Timer, 不需要显示调用, 调用 newTimeout 时, 会自动调用该方法

```

public void start() {
    //初始为WORKER_STATE_INIT, cas修改为WORKER_STATE_STARTED, 并启动worker线程
    //状态流转
    //如果初始化过了, 就启动worker
    //如果已经开始就跳过。如果关闭了会报错
    switch (WORKER_STATE_UPDATER.get(this)) {
        case WORKER_STATE_INIT:
            if (WORKER_STATE_UPDATER.compareAndSet(this, WORKER_STATE_INIT,
WORKER_STATE_STARTED)) {
                workerThread.start();
            }
        default:
            //已经启动过了, 什么都不做
    }
}

```

```

    }
    break;
case WORKER_STATE_STARTED:
    break;
case WORKER_STATE_SHUTDOWN:
    throw new IllegalStateException("cannot be started once stopped");
default:
    throw new Error("Invalid WorkerState");
}

```

```

// wait until the startTime is initialized by the worker.

//等待worker启动，并初始化startTime完成
while (startTime == 0) {
    try {
        startTimeInitialized.await();
    } catch (InterruptedException ignore) {
        // Ignore - it will be ready very soon.
    }
}
}
}

```

再看看 stop 方法:

//worker线程不能调用stop方法,也就是我们添加的Task中不能调用stop方法.

```
public Set stop() {
```

```

// worker线程不能停止时间轮，也就是加入的定时任务，不能调用这个方法。
// 不然会有恶意的定时任务调用这个方法而造成大量定时任务失效
if (Thread.currentThread() == workerThread) {
    throw new IllegalStateException(
        HashedWheelTimer.class.getSimpleName() +
        ".stop() cannot be called from " +
        TimerTask.class.getSimpleName());
}
//尝试更新为停止状态
if (!WORKER_STATE_UPDATER.compareAndSet(this, WORKER_STATE_STARTED,
WORKER_STATE_SHUTDOWN)) {
    // workerState can be 0 or 2 at this moment - let it always be 2.
    if (WORKER_STATE_UPDATER.getAndSet(this, WORKER_STATE_SHUTDOWN) !=
WORKER_STATE_SHUTDOWN) {
        INSTANCE_COUNTER.decrementAndGet();
        if (leak != null) {
            boolean closed = leak.close(this);
            assert closed;
        }
    }
}

return Collections.emptySet();
}

try {
    boolean interrupted = false;
    while (workerThread.isAlive()) {
        workerThread.interrupt();
        try {

```

```

        workerThread.join(100);
    } catch (InterruptedException ignored) {
        interrupted = true;
    }
}

if (interrupted) {
    Thread.currentThread().interrupt();
}
} finally {
    INSTANCE_COUNTER.decrementAndGet();
    if (leak != null) {
        boolean closed = leak.close(this);
        assert closed;
    }
}
return worker.unprocessedTimeouts();
}

```

重要的内部类，Worker：

```

private final class Worker implements Runnable {
    private final Set unprocessedTimeouts = new HashSet();

```

```

        private long tick;

        @Override
        public void run() {
            // 初始化startTime，所有的任务的截止时间都相对于这个startTime
            // Initialize the startTime.
            startTime = System.nanoTime();
            if (startTime == 0) {
                // We use 0 as an indicator for the uninitialized value here, so
                make sure it's not 0 when initialized.
                startTime = 1;
            }
            //唤醒start方法中的阻塞
            // Notify the other threads waiting for the initialization at start().
            startTimeInitialized.countDown();

            do {
                // 只要时间轮的状态为WORKER_STATE_STARTED，就循环转动tick
                // 返回值就是System.nanoTime() - startTime，也就是Timer启动后到这次tick，
                所过去的时间
                final long deadline = waitForNextTick();
                if (deadline > 0) {

                    //mask=2^n-1
                    // 获取tick对应的格子索引
                    int idx = (int) (tick & mask);
                    //移除取消了的任务
                    processCancelledTasks();

                    //取出对应的bucket（存放的就是任务）
                    HashedWheelBucket bucket =
                        wheel[idx];

```

```

        // 从任务队列中取出任务加入到对应的格子中，对应着看前面说过的newTimeout方法
        transferTimeoutsToBuckets();
        // 执行任务
        bucket.expireTimeouts(deadline);
        tick++;
    }
} while (WORKER_STATE_UPDATER.get(HashedWheelTimer.this) ==
WORKER_STATE_STARTED);

// Fill the unprocessedTimeouts so we can return them from stop()
method.
// 时间轮停止
// 清除所有格子中的任务，并加入到未处理任务列表，以供stop()方法返回
for (HashedWheelBucket bucket: wheel) {
    bucket.clearTimeouts(unprocessedTimeouts);
}

// 把还没来得及放进bucket的任务取出来，放到unprocessedTimeouts里
for (;;) {
    HashedWheelTimeout timeout = timeouts.poll();
    if (timeout == null) {
        break;
    }
    if (!timeout.isCancelled()) {
        unprocessedTimeouts.add(timeout);
    }
}
// 移除取消了的任务
processCancelledTasks();
}

// 把任务放进bucket里
private void transferTimeoutsToBuckets() {
    // transfer only max. 100000 timeouts per tick to prevent a thread to
    stale the workerThread when it just
    // adds new timeouts in a loop.
    // 最多放100000个
    for (int i = 0; i < 100000; i++) {
        HashedWheelTimeout timeout = timeouts.poll();
        if (timeout == null) {
            // all processed
            break;
        }
        // 还没有放入到格子中就取消了，直接略过
        if (timeout.state() == HashedWheelTimeout.ST_CANCELLED) {
            // was cancelled in the meantime.
            continue;
        }
        // 计算任务需要经过多少个tick
        long calculated = timeout.deadline / tickDuration;

        // 任务的轮数
        timeout.remainingRounds = (calculated - tick) / wheel.length;
    }
}

```

```

// 取计算出来的和当前tick的大值作为标准记性计算
// 这里是如果当前任务在队列里放了很久，以至于过了应该执行的时间

```



```

        // 那么就使用当前tick的位置，直接放到当前的格子中执行
        final long ticks = Math.max(calculated, tick); // Ensure we don't
        schedule for past.

        int stopIndex = (int) (ticks & mask);
        // 任务放到bucket里
        HashedWheelBucket bucket = wheel[stopIndex];
        bucket.addTimeout(timeout);
    }
}

private void processCancelledTasks() {
    for (;;) {
        HashedWheelTimeout timeout = cancelledTimeouts.poll();
        if (timeout == null) {
            // all processed
            break;
        }
        try {
            timeout.remove();
        } catch (Throwable t) {
            if (logger.isWarnEnabled()) {
                logger.warn("An exception was thrown while process a
cancellation task", t);
            }
        }
    }
}

/**
 * calculate goal nanoTime from startTime and current tick number,
 * then wait until that goal has been reached.
 * @return Long.MIN_VALUE if received a shutdown request,
 * current time otherwise (with Long.MIN_VALUE changed by +1)
 */

// 等待下一次tick
private long waitForNextTick() {
    long deadline = tickDuration * (tick + 1);

    for (;;) {
        final long currentTime = System.nanoTime() - startTime;

        // 计算需要sleep的时间，之所以加999999后再除10000000，是为了保证足够的sleep
        // 时间
        // 解决的问题，我举个例子，因为这里是ms，假设应该睡2.02ms，但是实际上只会睡
        2ms（时间精度问题，只能精确到ms）
        // 加999999只会，会变成3ms。
        // 实际上一句话就是，宁可多睡不少睡
        long sleepTimeMs = (deadline - currentTime + 999999) / 1000000;

        if (sleepTimeMs <= 0) {
            if (currentTime == Long.MIN_VALUE) {
                return -Long.MAX_VALUE;
            } else {
                return currentTime;
            }
        }
    }
}

```

```

        // Check if we run on windows, as if thats the case we will need
        // to round the sleepTime as workaround for a bug that only affect
        // the JVM if it runs on windows.
        //
        // See https://github.com/netty/netty/issues/356
        if (PlatformDependent.isWindows()) {
            sleepTimeMS = sleepTimeMS / 10 * 10;
        }

        try {
            Thread.sleep(sleepTimeMS);
        } catch (InterruptedException ignored) {
            if (WORKER_STATE_UPDATER.get(HashedWheelTimer.this) ==
WORKER_STATE_SHUTDOWN) {
                return Long.MIN_VALUE;
            }
        }
    }

    public Set<Timeout> unprocessedTimeouts() {
        return Collections.unmodifiableSet(unprocessedTimeouts);
    }
}

```

HashedWheelBucket 类:

```

private static final class HashedWheelBucket {
    // Used for the linked-list datastructure
    private HashedWheelTimeout head;
    private HashedWheelTimeout tail;

```

```

/**
 * Add {@link HashedWheelTimeout} to this bucket.
 */
// 添加操作，这里没有加锁，不会导致并发问题
// 为什么其实我们已经说过了，因为任务是在一个tick的时候由worker放进去的
public void addTimeout(HashedWheelTimeout timeout) {
    assert timeout.bucket == null;
    timeout.bucket = this;
    if (head == null) {
        head = tail = timeout;
    } else {
        tail.next = timeout;
        timeout.prev = tail;
        tail = timeout;
    }
}

/**
 * Expire all {@link HashedWheelTimeout}s for the given {@code deadline}.
 */
// 过期并执行格子中的到期任务，tick到该格子的时候，worker线程会调用这个方法
public void expireTimeouts(long deadline) {
    HashedWheelTimeout timeout = head;

```

```

// 循环所有这个槽位的任务
// process all timeouts
while (timeout != null) {
    HashedWheelTimeout next = timeout.next;
    // 只跑那些remainingRounds<=0的
    if (timeout.remainingRounds <= 0) {
        next = remove(timeout);
        if (timeout.deadline <= deadline) {
            timeout.expire();
        } else {
            // The timeout was placed into a wrong slot. This should
never happen.
            throw new IllegalStateException(String.format(
                "timeout.deadline (%d) > deadline (%d)",
timeout.deadline, deadline));
        }
    } else if (timeout.isCancelled()) {
        //取消的会在下次跑到这个槽位的时候移除任务
        next = remove(timeout);
    } else {
        timeout.remainingRounds --;
    }
    timeout = next;
}

// 移除一个节点，并返回被移除节点的下一个节点
public HashedWheelTimeout remove(HashedWheelTimeout timeout) {
    HashedWheelTimeout next = timeout.next;
    // remove timeout that was either processed or cancelled by updating the
linked-list
    if (timeout.prev != null) {
        timeout.prev.next = next;
    }
    if (timeout.next != null) {
        timeout.next.prev = timeout.prev;
    }

    if (timeout == head) {
        // if timeout is also the tail we need to adjust the entry too
        // timeout既是头也是尾的情况（链表里就一个任务）
        if (timeout == tail) {
            tail = null;
            head = null;
        } else {
            head = next;
        }
    } else if (timeout == tail) {
        // if the timeout is the tail modify the tail to be the prev node.
        tail = timeout.prev;
    }
    // null out prev, next and bucket to allow for GC.
    timeout.prev = null;
    timeout.next = null;
    timeout.bucket = null;
    timeout.timer.pendingTimeouts.decrementAndGet();
    return next;
}

```

```

/**
 * Clear this bucket and return all not expired / cancelled {@link
Timeout}s.
 */
// 清理过期（执行过或取消了的）任务，放到入参的set里
public void clearTimeouts(Set<Timeout> set) {
    for (;;) {
        HashedWheelTimeout timeout = pollTimeout();
        if (timeout == null) {
            return;
        }
        if (timeout.isExpired() || timeout.isCancelled()) {
            continue;
        }
        set.add(timeout);
    }
}

private HashedWheelTimeout pollTimeout() {
    HashedWheelTimeout head = this.head;
    if (head == null) {
        return null;
    }
    HashedWheelTimeout next = head.next;
    if (next == null) {
        tail = this.head = null;
    } else {
        this.head = next;
        next.prev = null;
    }

    // null out prev and next to allow for GC.
    head.next = null;
    head.prev = null;
    head.bucket = null;
    return head;
}
}

```

HashedWheelTimeout 类，任务的包装类，链表结构，负责保存deadline，轮数，等：

```
private static final class HashedWheelTimeout implements Timeout {
```

```

    private static final int ST_INIT = 0;
    private static final int ST_CANCELLED = 1;
    private static final int ST_EXPIRED = 2;
    private static final AtomicIntegerFieldUpdater<HashedWheelTimeout>
STATE_UPDATER =
        AtomicIntegerFieldUpdater.newUpdater(HashedWheelTimeout.class,
"state");

    private final HashedWheelTimer timer;
    private final TimerTask task;
    private final long deadline;

```

```

@SuppressWarnings({"unused", "FieldMayBeFinal",
"RedundantFieldInitialization" })
private volatile int state = ST_INIT;

// remainingRounds will be calculated and set by
Worker.transferTimeoutsToBuckets() before the
// HashedWheelTimeout will be added to the correct HashedWheelBucket.

// 离任务执行的轮数，当将次任务加入到格子中是计算该值，每过一轮，该值减一
// 这里就是我们说的，只应该执行那些，轮数为0的任务。
long remainingRounds;

// This will be used to chain timeouts in HashedWheelTimerBucket via a
double-linked-list.
// As only the workerThread will act on it there is no need for
synchronization / volatile.
HashedWheelTimeout next;
HashedWheelTimeout prev;

// The bucket to which the timeout was added
//bucket的引用
HashedWheelBucket bucket;

HashedWheelTimeout(HashedWheelTimer timer, TimerTask task, long deadline) {
    this.timer = timer;
    this.task = task;
    this.deadline = deadline;
}

@Override
public Timer timer() {
    return timer;
}

@Override
public TimerTask task() {
    return task;
}

@Override
public boolean cancel() {
    // 修改状态为ST_CANCELLED，会在下次tick时，在格子中移除
    // only update the state it will be removed from HashedWheelBucket on
next tick.
    if (!compareAndSetState(ST_INIT, ST_CANCELLED)) {
        return false;
    }
    // If a task should be canceled we put this to another queue which will
be processed on each tick.
    // So this means that we will have a GC latency of max. 1 tick duration
which is good enough. This way
    // we can make again use of our MpscLinkedQueue and so minimize the
locking / overhead as much as possible.

    // 加入到时间轮的待取消队列，并在每次tick的时候，从相应格子中移除。
    timer.cancelledTimeouts.add(this);
    return true;
}

```

```

void remove() {
    HashedWheelBucket bucket = this.bucket;
    if (bucket != null) {
        // 从格子中移除自身
        bucket.remove(this);
    } else {
        timer.pendingTimeouts.decrementAndGet();
    }
}

public boolean compareAndSetState(int expected, int state) {
    return STATE_UPDATER.compareAndSet(this, expected, state);
}

public int state() {
    return state;
}

@Override
public boolean isCancelled() {
    return state() == ST_CANCELLED;
}

@Override
public boolean isExpired() {
    return state() == ST_EXPIRED;
}

public void expire() {
    //更新状态为过期
    if (!compareAndSetState(ST_INIT, ST_EXPIRED)) {
        return;
    }

    try {
        task.run(this);
    } catch (Throwable t) {
        if (logger.isWarnEnabled()) {
            logger.warn("An exception was thrown by " +
                TimerTask.class.getSimpleName() + '.', t);
        }
    }
}

```

XXL-JOB原理--任务执行时间轮（六）

1、介绍

之前 XXL-JOB 任务执行是通过 Quartz来进行任务管理触发的，在之前的博客 [《Quartz任务调度框架-任务执行流程》](#) 我们进行了任务执行的流程介绍，目前 XXL-JOB 任务执行已经摒弃 Quartz 框架，目前通过时间轮方式来管理任务触发任务。

(1) 基本介绍

XXL-JOB 是一个轻量级分布式任务调度平台，主打特点是平台化，易部署，开发迅速、学习简单、轻量级、易扩展，代码仍在持续更新中。

- **调度中心**：任务调度控制台，平台自身并不承担业务逻辑，只是负责任务的统一管理和调度执行，并且提供任务管理平台
- **执行器**：负责接收“调度中心”的调度并执行，可直接部署执行器，也可以将执行器集成到现有业务项目中。通过将任务的调度控制和任务的执行解耦，业务使用只需要关注业务逻辑的开发。
- XXL-JOB 主要提供了任务的动态配置管理、任务监控和统计报表以及调度日志几大功能模块，支持多种运行模式和路由策略，可基于对应执行器机器集群数量进行简单分片数据处理。

(2) 框架源码及文档

源码地址：

- [GitHub](#)
- [码云](#)

文档地址：

- [中文文档](#)
- [英文文档](#)

(3) XXL-JOB的特性

- 1、**简单**：支持通过 Web 页面对任务进行 CRUD 操作，操作简单，一分钟上手；
- 2、**动态**：支持动态修改任务状态、启动/停止任务，以及终止运行中任务，即时生效；
- 3、**调度中心HA（中心式）**：调度采用中心式设计，调度中心自研调度组件并支持集群部署，可保证调度中心HA；
- 4、**执行器HA（分布式）**：任务分布式执行，任务“执行器”支持集群部署，可保证任务执行HA；
- 5、**注册中心**：执行器会周期性自动注册任务，调度中心将会自动发现注册的任务并触发执行。也支持手动录入执行器地址；
- 6、**弹性扩容缩容**：一旦有新执行器机器上线或者下线，下次调度时将会重新分配任务；
- 7、**路由策略**：执行器集群部署时提供丰富的路由策略，包括：**第一个、最后一个、轮询、随机、一致性HASH、最不经常使用、最近最久未使用、故障转移、忙碌转移**等；
- 8、**故障转移**：任务路由策略选择**故障转移**情况下，如果执行器集群中某一台机器故障，将会自动 Failover 切换到一台正常的执行器发送调度请求。
- 9、**阻塞处理策略**：调度过于密集执行器来不及处理时的处理策略，策略包括：**单机串行（默认）、丢弃后续调度、覆盖之前调度**；
- 10、**任务超时控制**：支持**自定义任务超时时间**，任务运行超时将会主动中断任务；
- 11、**任务失败重试**：支持**自定义任务失败重试次数**，当任务失败时将会按照预设的失败重试次数主动进行重试；其中分片任务支持分片粒度的失败重试；
- 12、**任务失败警告**：默认提供邮件方式失败告警，同时预留扩展接口，可方便的扩展短信、钉钉等告警方式；
- 13、**分片广播任务**：执行器集群部署时，任务路由策略选择**分片广播**情况下，一次任务调度将会广播触发集群中所有执行器执行一次任务，可根据分片参数开发分片任务；

- 14、**动态分片**：分片广播任务以执行器为维度进行分片，支持动态扩容执行器集群从而动态增加分片数量，协同进行业务处理；在进行大数据量业务操作时可显著提升任务处理能力和速度。
- 15、**事件触发**：除了Cron方式和任务依赖方式触发任务执行之外，支持基于事件的触发任务方式。调度中心提供触发任务单次执行的API服务，可根据业务事件灵活触发

2、任务执行

XXL-JOB 任务执行中启动了两个线程：

(1) 线程 scheduleThread 运行中不断的从任务表中查询 查询近 5000 毫秒(5秒)中要执行的任务，如果当前时间大于任务接下来要执行的时间则立即执行，否则将任务执行时间除以 1000 变为秒之后再与 60 求余添加到时间轮中。

(2) XXL-JOB 时间轮实现方式比较简单，就是一个 Map 结构数据，key值0-60，value是任务ID列表 Map<Integer, List> ringData

(3) 线程 ringThread 运行中不断根据当前时间求余从 时间轮 ringData 中获取任务列表，取出任务之后执行任务。

XXL-JOB 任务执行中启动了两个线程：

(1) 线程 scheduleThread 运行中不断的从任务表中查询 查询近 5000 毫秒(5秒)中要执行的任务，如果当前时间大于任务接下来要执行的时间则立即执行，否则将任务执行时间除以 1000 变为秒之后再与 60 求余添加到时间轮中。

(2) XXL-JOB 时间轮实现方式比较简单，就是一个 Map 结构数据，key值0-60，value是任务ID列表 Map<Integer, List> ringData

(3) 线程 ringThread 运行中不断根据当前时间求余从 时间轮 ringData 中获取任务列表，取出任务之后执行任务。

```
public class JobScheduleHelper {
    private static Logger logger = LoggerFactory.getLogger(JobScheduleHelper.class);
```

```
    private static JobScheduleHelper instance = new JobScheduleHelper();
    public static JobScheduleHelper getInstance(){
        return instance;
    }
    // 任务间隔大小
    public static final long PRE_READ_MS = 5000;    // pre read

    private Thread scheduleThread;
    private Thread ringThread;
    private volatile boolean scheduleThreadToStop = false;
    private volatile boolean ringThreadToStop = false;
    // 时间轮，key 0-60，value 任务ID列表，两个线程同时处理这个对象
    private volatile static Map<Integer, List<Integer>> ringData = new
    ConcurrentHashMap<>();

    public void start(){

        // schedule thread
        scheduleThread = new Thread(new Runnable() {
            @Override
            public void run() {

                try {
```



```

        TimeUnit.MILLISECONDS.sleep(5000 -
System.currentTimeMillis()%1000 );
    } catch (InterruptedException e) {
        if (!scheduleThreadToStop) {
            logger.error(e.getMessage(), e);
        }
    }
}
logger.info(">>>>>>> init xxl-job admin scheduler success.");

// pre-read count: treadpool-size * trigger-qps (each trigger cost
50ms, qps = 1000/50 = 20)
// 每次读取到任务个数
int preReadCount =
(XxlJobAdminConfig.getAdminConfig().getTriggerPoolFastMax() +
XxlJobAdminConfig.getAdminConfig().getTriggerPoolSlowMax()) * 20;

while (!scheduleThreadToStop) {

    // Scan Job
    long start = System.currentTimeMillis();

    Connection conn = null;
    Boolean connAutoCommit = null;
    PreparedStatement preparedStatement = null;

    boolean preReadSuc = true;
    try {

        conn =
XxlJobAdminConfig.getAdminConfig().getDataSource().getConnection();
        connAutoCommit = conn.getAutoCommit();
        conn.setAutoCommit(false);
        // 分布式下获取锁
        preparedStatement = conn.prepareStatement( "select * from
xxl_job_lock where lock_name = 'schedule_lock' for update" );
        preparedStatement.execute();

        // tx start

        // 1、pre read
        long nowTime = System.currentTimeMillis();
        // 查询 当前时间 + 5000 毫秒，就是接下来 5 秒要执行到任务
        List<XxlJobInfo> scheduleList =
XxlJobAdminConfig.getAdminConfig().getXxlJobInfoDao().scheduleJobQuery(nowTime +
PRE_READ_MS, preReadCount);
        if (scheduleList!=null && scheduleList.size()>0) {
            // 2、push time-ring
            for (XxlJobInfo jobInfo: scheduleList) {

                // time-ring jump
                // 如果当前时间大于要接下来执行到时间 + 5 秒
                if (nowTime > jobInfo.getTriggerNextTime() +
PRE_READ_MS) {

                    // 2.1、trigger-expire > 5s: pass && make next-
trigger-time

                    logger.warn(">>>>>>>>> xxl-job, schedule
misfire, jobId = " + jobInfo.getId());

```

```

        // 1、misfire match
        // 如果每次都有执行则立即触发执行
        MisfireStrategyEnum misfireStrategyEnum =
MisfireStrategyEnum.match(jobInfo.getMisfireStrategy(),
MisfireStrategyEnum.DO_NOTHING);
        if (MisfireStrategyEnum.FIRE_ONCE_NOW ==
misfireStrategyEnum) {
            // FIRE_ONCE_NOW 》 trigger

JobTriggerPoolHelper.trigger(jobInfo.getId(), TriggerTypeEnum.MISFIRE, -1, null,
null, null);

            logger.debug(">>>>>>>>> xxl-job, schedule
push trigger : jobId = " + jobInfo.getId() );
        }

        // 2、fresh next
        // 刷新接下来要执行时间
        refreshNextValidTime(jobInfo, new Date());

    } else if (nowTime > jobInfo.getTriggerNextTime()) {
        // 2.2、trigger-expire < 5s: direct-trigger &&
make next-trigger-time

        // 1、trigger
        // 如果当前时间大于接下来要执行到时间则立即触发执行
        JobTriggerPoolHelper.trigger(jobInfo.getId(),
TriggerTypeEnum.CRON, -1, null, null, null);
        logger.debug(">>>>>>>>> xxl-job, schedule push
trigger : jobId = " + jobInfo.getId() );

        // 2、fresh next
        // 刷新下次执行时间
        refreshNextValidTime(jobInfo, new Date());

        // next-trigger-time in 5s, pre-read again
        if (jobInfo.getTriggerStatus()==1 && nowTime +
PRE_READ_MS > jobInfo.getTriggerNextTime()) {

            // 1、make ring second
            // 如果接下来 5 秒内还执行则直接放到时间轮中
            int ringSecond = (int)
((jobInfo.getTriggerNextTime()/1000)%60);

            // 2、push time ring
            pushTimeRing(ringSecond, jobInfo.getId());

            // 3、fresh next
            // 刷新下次执行时间
            refreshNextValidTime(jobInfo, new
Date(jobInfo.getTriggerNextTime()));

        }

    } else {
        // 2.3、trigger-pre-read: time-ring trigger &&
make next-trigger-time

        // 1、make ring second

```

```

        // 任务还没有到执行时间则直接放到时间轮中
        int ringSecond = (int)
((jobInfo.getTriggerNextTime()/1000)%60);

        // 2、push time ring
        pushTimeRing(ringSecond, jobInfo.getId());

        // 3、fresh next
        // 刷新下次执行时间
        refreshNextValidTime(jobInfo, new
Date(jobInfo.getTriggerNextTime()));

    }

}

// 3、update trigger info
for (XxlJobInfo jobInfo: scheduleList) {
    // 更新任务信息

XxlJobAdminConfig.getAdminConfig().getXxlJobInfoDao().scheduleUpdate(jobInfo);
}

} else {
    preReadSuc = false;
}

// tx stop

```

```

    } catch (Exception e) {
        if (!scheduleThreadToStop) {
            logger.error(">>>>>>>>>> xxl-job,
JobsScheduleHelper#scheduleThread error:{})", e);
        }
    } finally {

        // commit
        if (conn != null) {
            try {
                conn.commit();
            } catch (SQLException e) {
                if (!scheduleThreadToStop) {
                    logger.error(e.getMessage(), e);
                }
            }
            try {
                conn.setAutoCommit(connAutoCommit);
            } catch (SQLException e) {
                if (!scheduleThreadToStop) {
                    logger.error(e.getMessage(), e);
                }
            }
            try {
                conn.close();
            } catch (SQLException e) {
                if (!scheduleThreadToStop) {
                    logger.error(e.getMessage(), e);
                }
            }
        }
    }
}

```

```

    }
    }
}

// close PreparedStatement
if (null != preparedStatement) {
    try {
        preparedStatement.close();
    } catch (SQLException e) {
        if (!scheduleThreadToStop) {
            logger.error(e.getMessage(), e);
        }
    }
}
}

long cost = System.currentTimeMillis()-start;

```

```

// wait seconds, align second
if (cost < 1000) { // scan-overtime, not wait
    try {
        // pre-read period: success > scan each second; fail >
skip this period;
        TimeUnit.MILLISECONDS.sleep((preReadSuc?
1000:PRE_READ_MS) - System.currentTimeMillis()%1000);
    } catch (InterruptedException e) {
        if (!scheduleThreadToStop) {
            logger.error(e.getMessage(), e);
        }
    }
}

    }

    logger.info(">>>>>>>>> xxl-job, JobsScheduleHelper#scheduleThread
stop");
}

});
scheduleThread.setDaemon(true);
scheduleThread.setName("xxl-job, admin JobsScheduleHelper#scheduleThread");
scheduleThread.start();

```

```

// ring thread
ringThread = new Thread(new Runnable() {
    @Override
    public void run() {

        // align second
        try {
            TimeUnit.MILLISECONDS.sleep(1000 -
System.currentTimeMillis()%1000 );
        } catch (InterruptedException e) {
            if (!ringThreadToStop) {
                logger.error(e.getMessage(), e);
            }
        }
    }
}

```

```

        while (!ringThreadToStop) {

            try {
                // second data
                List<Integer> ringItemData = new ArrayList<>();
                int nowSecond = Calendar.getInstance().get(Calendar.SECOND);
                // 避免处理耗时太长，跨过刻度，向前校验一个刻度；
                // 获取最近1秒和 2秒要执行到任务
                for (int i = 0; i < 2; i++) {
                    List<Integer> tmpData = ringData.remove( (nowSecond+60-
i)%60 );

                    if (tmpData != null) {
                        ringItemData.addAll(tmpData);
                    }
                }

                // ring trigger
                logger.debug(">>>>>>>>>> xxl-job, time-ring beat : " +
nowSecond + " = " + Arrays.asList(ringItemData) );
                if (ringItemData.size() > 0) {
                    // do trigger
                    for (int jobId: ringItemData) {
                        // do trigger
                        // 执行任务
                        JobTriggerPoolHelper.trigger(jobId,
TriggerTypeEnum.CRON, -1, null, null, null);
                    }
                    // clear
                    ringItemData.clear();
                }
            } catch (Exception e) {
                if (!ringThreadToStop) {
                    logger.error(">>>>>>>>>> xxl-job,
JobsScheduleHelper#ringThread error:{", e);
                }
            }

            // next second, align second
            try {
                TimeUnit.MILLISECONDS.sleep(1000 -
System.currentTimeMillis()%1000);
            } catch (InterruptedException e) {
                if (!ringThreadToStop) {
                    logger.error(e.getMessage(), e);
                }
            }

            logger.info(">>>>>>>>>> xxl-job, JobsScheduleHelper#ringThread
stop");
        }
    });
    ringThread.setDaemon(true);
    ringThread.setName("xxl-job, admin JobsScheduleHelper#ringThread");
    ringThread.start();
}

private void refreshNextValidTime(XxlJobInfo jobInfo, Date fromTime) throws
Exception {

```

```

Date nextValidTime = generateNextValidTime(jobInfo, fromTime);
if (nextValidTime != null) {
    jobInfo.setTriggerLastTime(jobInfo.getTriggerNextTime());
    jobInfo.setTriggerNextTime(nextValidTime.getTime());
} else {
    jobInfo.setTriggerStatus(0);
    jobInfo.setTriggerLastTime(0);
    jobInfo.setTriggerNextTime(0);
    logger.warn(">>>>>>>>> xxl-job, refreshNextValidTime fail for job:
jobId={}, scheduleType={}, scheduleConf={}",
        jobId(), jobInfo.getScheduleType(),
        jobInfo.getScheduleConf());
}
}

private void pushTimeRing(int ringSecond, int jobId){
    // push async ring
    List<Integer> ringItemData = ringData.get(ringSecond);
    if (ringItemData == null) {
        ringItemData = new ArrayList<Integer>();
        ringData.put(ringSecond, ringItemData);
    }
    ringItemData.add(jobId);

    logger.debug(">>>>>>>>> xxl-job, schedule push time-ring : " + ringSecond
+ " = " + Arrays.asList(ringItemData) );
}

public void toStop(){

    // 1. stop schedule
    scheduleThreadToStop = true;
    try {
        TimeUnit.SECONDS.sleep(1); // wait
    } catch (InterruptedException e) {
        logger.error(e.getMessage(), e);
    }
    if (scheduleThread.getState() != Thread.State.TERMINATED){
        // interrupt and wait
        scheduleThread.interrupt();
        try {
            scheduleThread.join();
        } catch (InterruptedException e) {
            logger.error(e.getMessage(), e);
        }
    }

    // if has ring data
    boolean hasRingData = false;
    if (!ringData.isEmpty()) {
        for (int second : ringData.keySet()) {
            List<Integer> tmpData = ringData.get(second);
            if (tmpData!=null && tmpData.size()>0) {
                hasRingData = true;
                break;
            }
        }
    }
}

```

```

        if (hasRingData) {
            try {
                TimeUnit.SECONDS.sleep(8);
            } catch (InterruptedException e) {
                logger.error(e.getMessage(), e);
            }
        }

        // stop ring (wait job-in-memory stop)
        ringThreadToStop = true;
        try {
            TimeUnit.SECONDS.sleep(1);
        } catch (InterruptedException e) {
            logger.error(e.getMessage(), e);
        }
        if (ringThread.getState() != Thread.State.TERMINATED){
            // interrupt and wait
            ringThread.interrupt();
            try {
                ringThread.join();
            } catch (InterruptedException e) {
                logger.error(e.getMessage(), e);
            }
        }

        logger.info(">>>>>>>>>>>> xxl-job, JobScheduleHelper stop");
    }

```

```

// ----- tools -----
public static Date generateNextValidTime(XxlJobInfo jobInfo, Date fromTime)
throws Exception {
    ScheduleTypeEnum scheduleTypeEnum =
        ScheduleTypeEnum.match(jobInfo.getScheduleType(), null);
    if (ScheduleTypeEnum.CRON == scheduleTypeEnum) {
        Date nextValidTime = new
        CronExpression(jobInfo.getScheduleConf()).getNextValidTimeAfter(fromTime);
        return nextValidTime;
    } else if (ScheduleTypeEnum.FIX_RATE == scheduleTypeEnum /*||
        ScheduleTypeEnum.FIX_DELAY == scheduleTypeEnum*/) {
        return new Date(fromTime.getTime() +
        Integer.valueOf(jobInfo.getScheduleConf())*1000 );
    }
    return null;
}

```

通过以上分析，我们发现 XXL-JOB 通过一个简单到时间轮就可以完成一个任务到倒计时执行操作。

参考

<https://www.likecs.com/show-204434429.html>
<http://www.cs.columbia.edu/~nahum/w6998/papers/sosp87-timing-wheels.pdf>
<https://blog.csdn.net/xinzhongtianxia/article/details/86221241>
https://blog.csdn.net/m0_37039331/article/details/87401758
<https://blog.csdn.net/qq924862077/article/details/112550085>
<https://baijiahao.baidu.com/s?id=1714290103234167995>
<https://www.cnblogs.com/smilelce/p/11156412.html>
<https://blog.csdn.net/bz120413/article/details/122107790>
<https://blog.csdn.net/Javaesandyou/article/details/123918852>
<https://blog.csdn.net/Javaesandyou/article/details/123918852>
<https://blog.csdn.net/FreeeLinux/article/details/54897192>
https://blog.csdn.net/weixin_41605937/article/details/121972371

推荐阅读：

《尼恩Java面试宝典》

《Springcloud gateway 底层原理、核心实战 (史上最全)》

《Flux、Mono、Reactor 实战 (史上最全) 》

《Sentinel (史上最全) 》

《Nacos (史上最全)》

《分库分表 Sharding-JDBC 底层原理、核心实战 (史上最全) 》

《Clickhouse 超底层原理 + 高可用实操 (史上最全) 》

《Redis 集群 实操 (史上最全、5w字长文) 》

《一文搞定：SpringBoot、SLF4j、Log4j、Logback、Netty之间混乱关系 (史上最全) 》

《红黑树 (图解 + 秒懂 + 史上最全) 》

《分布式事务 (秒懂) 》

《缓存之王：Caffeine 源码、架构、原理 (史上最全，10W字 超级长文) 》

《缓存之王：Caffeine 的使用 (史上最全) 》

《Java Agent 探针、字节码增强 ByteBuddy (史上最全) 》

《Docker原理 (图解+秒懂+史上最全) 》

《Redis分布式锁 (图解 - 秒懂 - 史上最全) 》

《Zookeeper 分布式锁 - 图解 - 秒懂》

《Zookeeper Curator 事件监听 - 10分钟看懂》

《Netty 粘包 拆包 | 史上最全解读》

《Netty 100万级高并发服务器配置》

《Springcloud 高并发 配置（一文全懂）》

版权声明：本文为CSDN博主「40岁资深老架构师尼恩」的原创文章，遵循CC 4.0 BY-SA版权协议，转载请附上原文出处链接及本声明。

原文链接：<https://blog.csdn.net/crazymakercircle/article/details/128265067>

[TCP协议详解\(史上最全\)](#)

[clickhouse 超底层原理 + 高可用实操（史上最全）](#)

[nacos高可用（图解+秒懂+史上最全）](#)

[队列之王：Disruptor 原理、架构、源码 一文穿透](#)

[环形队列、条带环形队列 Striped-RingBuffer（史上最全）](#)

[一文搞定：SpringBoot、SLF4j、Log4j、Logback、Netty之间混乱关系（史上最全）](#)

[单例模式（史上最全）](#)

硬核推荐：尼恩Java硬核架构班

又名疯狂创客圈社群 VIP

详情：<https://www.cnblogs.com/crazymakercircle/p/9904544.html>



尼恩java
硬核架构班

已经发布

- 《高能性能RPC的基础实操之：从0到1开始IM撸一个IM》
- 《分布式高能性能RPC的基础实操之：千万级用户分布式IM实操-含简历指导》
- 《亿级用户超高并发秒杀实操-含简历指导》
亮点：助力小伙伴搞定70W年薪，N个涨薪50%，2023春招面试涨薪神器
- 《横扫全网，工业级elasticsearch底层原理与高并发、高可用架构实操》
亮点：40岁老架构师细致解读，处处透着分布式、高性能中间件的原理和精髓
- 《第1部曲：超级底层：葵花宝典（高性能秘籍）__架构师视角解读OS操作系统》
亮点：大制作解读OS操作系统，并揭秘mmap、pagecache、zerocopy等底层的底层原理
2023春招面试涨薪大神器
- 《Rocketmq视频第2部曲：横扫全网工业级 rocketmq 高可用（HA）底层原理和实操》
亮点：起底式、绞杀式解读 rocketmq如何保障消息的可靠性？
- 《Rocketmq视频第3部曲：超级内功篇、横扫全网 rocketmq 源码学习以及3高架构模式解读》
亮点：大制作解读 Rocketmq源码以及3高架构模式，助力大家内力猛增
- 《Rocketmq视频第4部曲：10Wqps消息推送中台架构、设计、编码、测试实操》
亮点：Netty实操、分库分表实操、Rocketmq工业级使用实操
- 《架构师内功篇：横扫全网 netty 高性能、高并发架构 底层原理、源码学习》
- 《架构师实操篇：redis cluster 工业级高可用实操》
- 《架构师实操篇：100W级别QPS日志平台实操》
- 《彻底穿透：skywalking 源码(代表链路跟踪)+Java agent+bytebuddy 探针》

规划中

《架构师实操篇：基于netty 手写 rpc 框架- 参考 dubbo、seata rpc框架》

《架构师实操篇：go语言学习，以及基于 go 手写 rpc 框架》

《架构师实操篇：千万级任务调度平台 架构与实操- 基于尼恩17年的亿级搜索项目》

《架构师实操篇：工业级 亿级文档搜索 平台 架构与实操- 基于尼恩17年的亿级搜索项目》

特色

会员制

提供技术方向指导，
职业生涯指导，少躺坑，少弯路

简历指导

这个很重要，
对于挪窝涨薪来说

实操性

以上项目，都是老架构师
在生产上实操过的项目

非水货

40岁老架构师，不是水货架构师
《Java高并发三部曲》为证

架构班（社群 VIP）的起源：

最初的视频，主要是给读者加餐。很多的读者，需要一些高质量的实操、理论视频，所以，我就围绕书，和底层，做了几个实操、理论视频，然后效果还不错，后面就做成迭代模式了。

架构班（社群 VIP）的功能：

提供高质量实操项目整刀真枪的架构指导、快速提升大家的：

- 开发水平
- 设计水平
- 架构水平

弥补业务中 CRUD 开发短板，帮助大家尽早脱离具备 3 高能力，掌握：

- 高性能
- 高并发
- 高可用

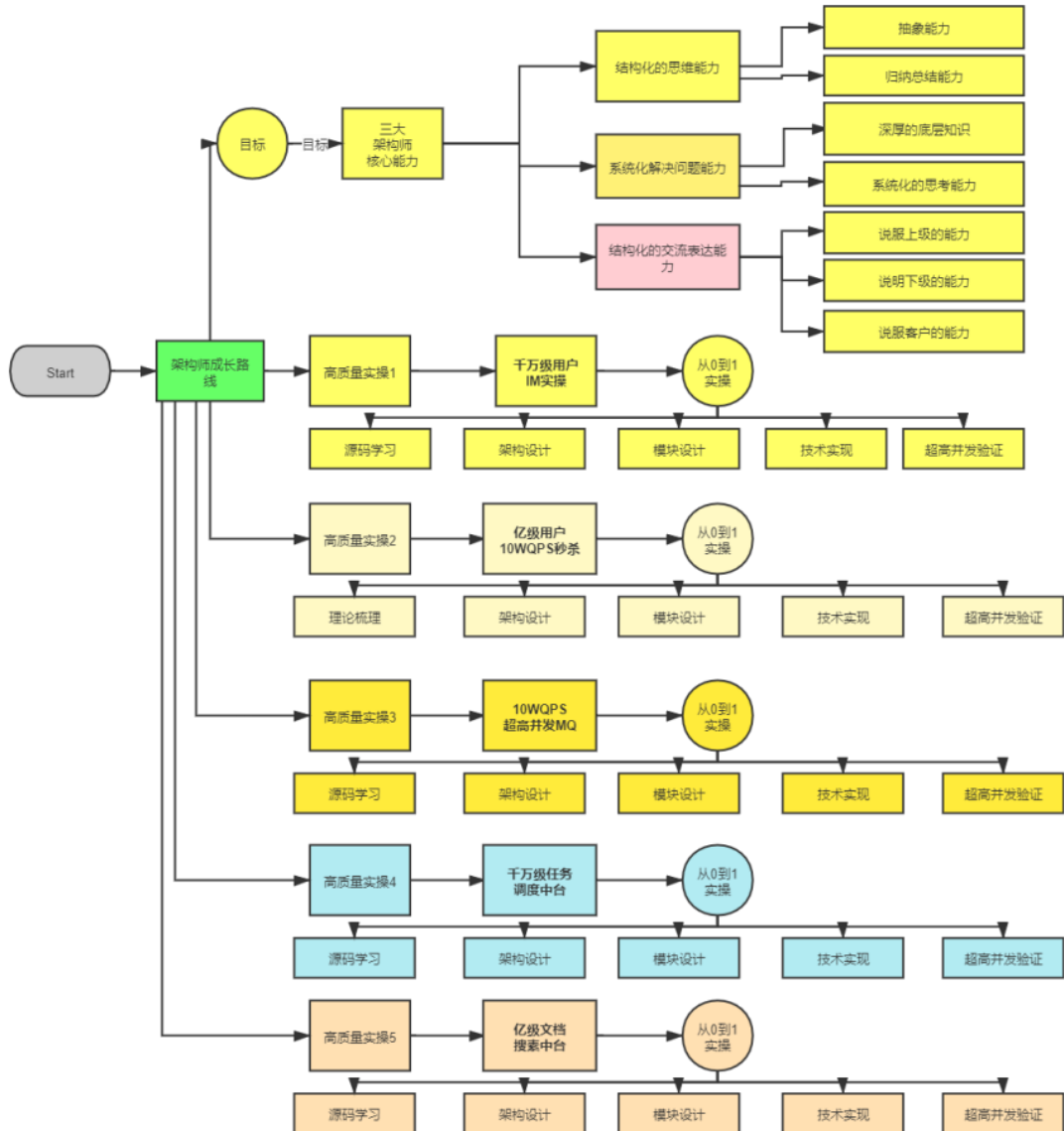
作为一个高质量的架构师成长、人脉社群，把所有的卷王聚焦起来，一起卷：

- 卷高并发实操
- 卷底层原理
- 卷架构理论、架构哲学
- 最终成为顶级架构师，实现人生理想，走向人生巅峰

架构班（社群 VIP）的目的：

- 高质量的实操，大大提升简历的含金量，吸引力，增强面试的召唤率
- 为大家提供九阳真经、葵花宝典，快速提升水平
- 进大厂、拿高薪
- 一路陪伴，提供助学视频和指导，辅导大家成为架构师
- 自学为主，和其他卷王一起，卷高并发实操，卷底层原理、卷大厂面试题，争取狠卷 3 月成高手，狠卷 3 年成为顶级架构师

N 个超高并发实操项目：简历压轴、个顶个精彩



【样章】第 17 章:横扫全网Rocketmq 视频第 2 部曲: 工业级 rocketmq 高可用(HA) 底层原理和实操

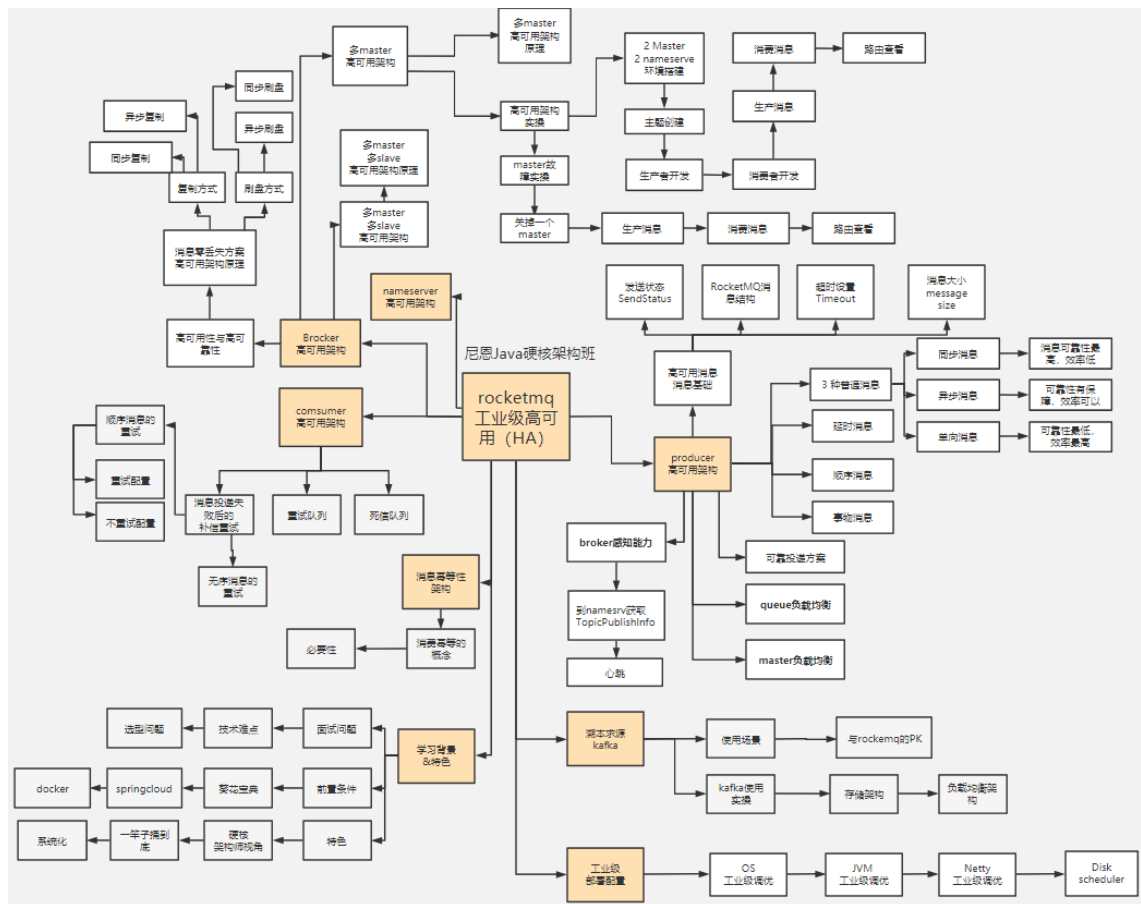
工业级 rocketmq 高可用底层原理, 包含: 消息消费、同步消息、异步消息、单向消息等不同消息的底层原理和源码实现; 消息队列非常底层的主从复制、高可用、同步刷盘、异步刷盘等底层原理。

工业级 rocketmq 高可用底层原理和搭建实操, 包含: 高可用集群的搭建。

解决以下难题:

- 1、技术难题: RocketMQ 如何最大限度的保证消息不丢失的呢? RocketMQ 消息如何做到高可靠投递?
- 2、技术难题: 基于消息的分布式事务, 核心原理不理解
- 3、选型难题: kafka or rocketmq , 该娶谁?

下图链接: <https://www.proceson.com/view/6178e8ae0e3e7416bde9da19>



成功案例：2 年翻 3 倍，35 岁卷王成功转型为架构师

详情: <http://topcoder.cloud/forum.php?mod=forumdisplay&fid=43&page=1>

最新 最后发表 热门 精华

成功案例: [1057号卷王] 3年小伙拿到外企offer, 薪酬涨了200%

1 卷王1号 超级版主 前天 17:41

成功案例: [645号卷王] 4年经验卷王逆袭, 被毕业后, 反涨24W

1 卷王1号 超级版主 2022-9-21

成功案例: [878号卷王] 小伙8年经验, 年薪60W

1 卷王1号 超级版主 2022-8-13

年薪70W案例: 通过尼恩的指导, 小伙伴年薪从40W涨到70W

1 卷王1号 超级版主 2022-2-11

成功案例: [493号卷王] 5年小伙拿满意offer, 就业寒冬季逆涨30%

1 卷王1号 超级版主 前天 17:43

成功案例: [250号卷王] 就业极寒时代, 收offer 涨25%

1 卷王1号 超级版主 前天 17:38

成功案例: [612号卷王] 就业极寒时代, 从外包到自研

1 卷王1号 超级版主 前天 17:15

成功案例: [913号卷王] 热烈祝贺6年经验卷王, 年薪40W

1 卷王1号 超级版主 2022-9-21

成功案例: [959号卷王] 4年经验卷王, 喜获百度、Boss直聘等N个优质offer, 最高涨100%

1 卷王1号 超级版主 2022-9-21

成功案例: [529号卷王] 5年经验卷王喜收2大offer, 最高涨5K

1 卷王1号 超级版主 2022-9-21

成功案例: [811号卷王] 热烈祝贺7年经验卷王, 薪酬涨30%

1 卷王1号 超级版主 2022-9-21

成功案例: [287号卷王] 不惧大寒潮, 卷王逆市收4 offer, 涨30%, 可喜可贺

1 卷王1号 超级版主 2022-5-30

成功案例: [1002号卷王] 5月份“被毕业”, 改简历后, 斩获顶级央企Offer, 涨薪7000+

1 卷王1号 超级版主 2022-7-5

成功案例: [7号卷王] 热烈祝贺小伙伴涨薪120%

① 卷王1号 超级版主 2022-8-13

成功案例: [134号卷王] 大三小伙卷1年, 斩获顶级央企Offer, 成功逆袭

① 卷王1号 超级版主 2022-7-6

成功案例: [1008号卷王] 5年经验卷王收42W offer, 月涨8000, 可喜可贺

① 卷王1号 超级版主 2022-5-30

成功案例: [453号卷王] 非全日制 6年卷王喜提3 offer, 年薪30W, 可喜可贺

① 卷王1号 超级版主 2022-5-21

成功案例: [924号卷王] 6年卷王喜提4 offer, 最高涨薪9000, 可喜可贺

① 卷王1号 超级版主 2022-5-21

成功案例: [15号卷王] 4年卷王入职 微软, 涨薪50%, 可喜可贺

① 卷王1号 超级版主 2022-5-12

成功案例: [527号卷王] 4年卷王喜提2 offer, 涨薪50%, 可喜可贺

① 卷王1号 超级版主 2022-5-13

成功案例: [788号卷王] 3年卷王喜提优质Offer, 涨薪60%

① 卷王1号 超级版主 2022-5-11

成功案例: 热烈祝贺: 非全日制卷王, 喜提2个心仪offer, 面3家过2家

① 卷王1号 超级版主 2022-4-21

成功案例: [693号卷王] 二线城市6年卷王喜提4大优质Offer, 含央企offer, 最高薪酬35W

① 卷王1号 超级版主 2022-4-16

成功案例: [85号卷王] 双非2本小伙, 春招大捷, 喜提9个offer, 最高薪酬近30万

① 卷王1号 超级版主 2022-4-14

成功案例: [741号卷王] 卷王逆袭! 6年小伙从很少面试机会到搞定35K*14薪Offer

① 卷王1号 超级版主 2022-4-12

成功案例: [642号卷王] 热烈祝贺, 6年卷王喜提优质国企offer

① 卷王1号 超级版主 2022-4-7

成功案例: [796号卷王] 热烈祝贺, 36岁卷王喜提52万优质offer

① 卷王1号 超级版主 2022-3-25

☐ 成功案例: [15号卷王] 小伙卷1年, 涨薪9K+, 喜收ebay等多个优质offer

① 卷王1号 超级版主 2022-3-24

☐ 成功案例: [821号卷王] 小伙狠卷3个月, 喜提10多个offer

① 卷王1号 超级版主 2022-3-21

☐ 成功案例: [736号卷王] 3年半经验收22k offer, 但是小伙志存高远, 冲击25k+

① 卷王1号 超级版主 2022-3-20

☐ 成功案例: 热烈祝贺1群小卷王offer拿到手软, 甚至拒了阿里offer

① 卷王1号 超级版主 2022-3-16

☐ 简历案例: 简历一改, 腾讯的邀请就来了! 热烈祝贺, 小伙收到一大堆面试邀请

① 卷王1号 超级版主 2022-3-10

☐ 成功案例: 祝贺我国两大超级卷王, 一个过了阿里HR面, 一个过了阿里2面

① 卷王1号 超级版主 2022-3-10

☐ 成功案例: 小伙伴php转Java, 卷1.5年Java, 涨薪50%, 喜收多个优质offer

① 卷王1号 超级版主 2022-3-10

☐ 成功案例: 4年小伙狠卷半年, 拿到 移动、京东 两大顶级offer

 尼恩 超级版主 2022-3-5

☐ 成功案例: [267号卷王] 助力3年经验卷王, 拿到蜂巢的17k x 14薪的offer

① 卷王1号 超级版主 2022-2-27

☐ 成功案例: [143号卷王] 二本院校00后卷神, 毕业没到一年跳到字节, 年薪45W

① 卷王1号 超级版主 2022-2-27

☐ 成功案例: [494号卷王] 尼恩分布式事务助力卷王拿到 中信银行offer

① 卷王1号 超级版主 2022-2-27

☐ 成功案例: [76号卷王] 2线城市卷王, 狠卷1.5年, 喜收22K offer

① 卷王1号 超级版主 2022-2-27

☐ 成功案例: [429号卷王] 小伙伴在社群卷5个月, 涨8k+

① 卷王1号 超级版主 2022-2-27

☐ 成功案例: [154号卷王] 双非学校毕业卷王, 连拿 京东到家&滴滴 两个大厂 Offer

① 卷王1号 超级版主 2022-2-27

☐ 成功案例: [232号卷王] 涨薪10K, 继续卷向食物链顶端

① 卷王1号 超级版主 2022-2-27

☐ 成功案例: 狠卷1年技术, 喜收 腾讯、阿里、微软三大Offer, 最高年薪56W

① 卷王1号 超级版主 2022-2-27

☐ 成功案例: [449号卷王] 应届毕业生卷王喜收 滴滴offer, 年薪33W

① 卷王1号 超级版主 2022-2-27

☐ 成功案例: [551号卷王] 小伙伴学完后, 成功进入大厂, 并且推荐自己的朋友加VIP学习

① 卷王1号 超级版主 2022-2-10

☐ 成功案例: [214号卷王] 助力2年经验卷王, 成功拿到17K月薪

① 卷王1号 超级版主 2022-2-10

☐ 成功案例: [92号卷王] 课程实操助力社群小伙伴喜收 喜马拉雅Offer

① 卷王1号 超级版主 2022-2-10

☐ 成功案例: 社群卷王小伙伴成功过了滴滴三面 获滴滴Offer

① 卷王1号 超级版主 2022-2-10

☐ [612号卷王]滴滴小伙伴, 蹲点考察半年, 觉得靠谱后加入 疯狂创客圈

① 卷王1号 超级版主 2022-2-10

☐ 成功案例: [732号卷王] 尼恩助力3年经验卷王收获 京东offer, 年薪35W

① 卷王1号 超级版主 2022-2-27

☐ 成功案例: [558号卷王] 2年经验卷王, 喜收 网易和阿里子公司两个优质offer

① 卷王1号 超级版主 2022-2-27

☐ 成功案例: [569号卷王] 双非应届生卷王, 喜收字节跳动实习offer

① 卷王1号 超级版主 2022-2-25

☐ 成功案例: [420号卷王] 狠卷1年, 卷王涨薪80%, 涨薪12000元!

① 卷王1号 超级版主 2022-2-25

☐ 成功案例: [76号卷王] 通过尼恩1年半的指导, 专科学历小伙伴从0.8K涨到22K

① 卷王1号 超级版主 2022-2-10

简历优化后的成功涨薪案例（VIP 含免费简历优化）

9年小伙伴拿到年薪90W offer

9月11日改简历 11月29日晒offer

秘诀:
简历指导+ 狠狠卷

薪资高 稳

这个是你微调的

主要的工作是啥?

省路号的地方, 需要你再补充一点

提升了自己的能力, 就不用怕

易所 关于数字货币的

也有打盹的时候, 该裁员, 照样一个不少

上面你留着这个就行

年包比 多 19w

这么多

估计有 70 万

那你不是有 90 个 W?

给我 68

就是吗

Java 开发 - 9 年 - 修改.docx 34.1 KB

小伙8年经验 年薪60W

7月12日改简历 8月10日晒offer

秘诀:
改简历+ 狠狠卷

涨幅多少呀?

或者, 幅度多少呀?

之前 36*15, 现在这个 39*15

今年行情不太好, 还有一些 offer 基本都是持平, 没降薪的。

OK

这个马上来

刚在指导简历

哈哈

恭喜呀

这次找工作, 您的简历写得超到位了。

我这次面试基本都是看的都是咱们课程的 面试 题。

OK

那就上午 11 点左右吧

好的

6年小伙伴 年薪40W

9月6日改简历 9月21日晒offer

秘诀:
简历指导+ 狠狠卷

张这多 20K

今年这行情, 也算可以了

总包多少呀, 让我也围观一下

大概 40w 吧

谢谢恩师的指导和鼓励

是在深圳

深圳的行情尤其难

能有面试电话就不错了

是的

太牛啦

哈哈哈哈哈, 恩师的鼓励指导也很重要

给你前面调整了一下

简历 - Java - 6 年.docx 24.7 KB

简历 - Java - 6 年(1).docx 24.5 KB

5年小伙喜提3个offer 年薪35个W

5月22日改简历 11月29日晒offer

秘诀:
简历指导+ 狠狠卷

恩恩老师晚上好, 汇报下那说的 offer 情况, 最近面试收到了三个 offer, 两个是平薪, 一个是跨境电商公司的 offer, 涨幅暂时不到 20%, 通过这三次面试也让我更加清楚自己距离高级开发还有一点点距离, 还要再多卷卷才能突破。

最后结合自己的情况, 先选择去跨境电商的公司再往下

之前是 20K*13.5, 跨境电商这个是 23K* (14-15)

恭喜恭喜

9 9 9

就业寒冬, 能拿到 3 个 offer, 已经很不错了, 已经是高手了

很多小伙伴, 面试电话一个都不接到, 简历海投 7000 份, 只收到 3 次面试机会, 没有一个机会拿到最终 offer

您这个年薪, 算下来也有 35W 了吧

年终奖部分还要确认下, 大部分人听说只有两个月

这个时间点, 拿到这个水平, 挺不错的啦

持续加油卷哈

嘿嘿! 看看明年自己有没有能力冲击离开

把你视频都吃透

辛苦老师再帮忙指导下哈

我报好号, 回头找你哈

OK

1.5年小伙搞定15K offer

就业寒冬涨100%

5月7日改简历 11月21日晒offer

秘诀:
简历指导+ 狠狠卷

他那个不止再修修嘛。我才是两位。原先7k, 现在15k

那也很牛

都翻倍了, 都是牛人

正常就30%

好的

晚上指导你改

推送中台需要加不

我还没做完, 准备八脱文新时没时间来搞这些, 入职了会开始搞

OK

还有其他的项目吗

一个项目, 太少啦

那种练习项目也行

其实我工作一年, 你

那你就更牛逼啦

一年经验, 搞定15k offer, 舍你其谁

卷王逆袭成功案例

6年小伙从很少面试机会到搞定35K*14薪

3月5日改简历 4月11日拿offer

一个月拿到了理想的offer

面试邀请少 小伙很苦恼

理想很丰满 目标35K

面试法宝 rocketmq四部曲

6年 经验小伙伴喜收25K offer

3月12日改简历 12月1日晒offer

秘诀:
简历指导+ 狠狠卷

咱们以这个项目为模板改哈

项目有多少人, 你带领多少人?

你工作几年了?

6年了

改简历那, 20 涨到 25 k, 高级开发, P7 带的后端团队

任务重, 道路远, 并不是没有方向

7年经验卷王薪酬涨30%

7月11日改简历 9月1日晒offer

秘诀:
改简历+ 狠狠卷

只要本事好, 拿 offer 比较容易的

喊了, 但是我流水太差了。。。我这是靠补贴, 银行不认这玩意嘛, 只能在工资的基础上加到 30% 封顶 (特别优秀除外, 我只是不错。。。)

入职了, 最终并不搞补贴嘛, 还差旧的大半, 只涨了 30%

恭喜一下

现在不比往年

能涨 30% 是大牛

拿到 offer 都算大牛

现在很多人发几百发, 这个电话都没有

你已经很牛啦

4年经验卷王逆袭 被毕业后，反涨24W

7月改简历 8月30日晒offer

秘诀:
改简历 + 狠卷

这就是你的简历
差得太多啦
老哥 我八月十号开始找工作，今天已入职了
能涨24W
原因大概是啥?
很多小伙伴，面试机会都没有
这个地方的要这样写
项目被终止
方便语音沟通不

尼恩 你这么指导，非常重要
老哥 我八月十号开始找工作，今天已入职了
现金基本持平，股票 +24W
总计涨了多少呢
股票这个吧只能到手了才算
也不错啦
继续跟着你学技术

小伙5月份"被毕业"，改简历后 斩获顶级央企Offer 涨薪7000+

5月29日改简历 7月5日晒offer

秘诀:
简历指导 + 狠卷3高

快速看书，就能不求甚解，把自读和场景大概一下，然后重点的地方，用到的地方，再去回顾
我被"毕业"了
这周末或下周找你改一下简历
毕业没有关系
ok，发我吧
R行业，就来说去，太复杂啦
嗯，其实有点心理准备
不太会写简历
我拿到手写的offer了
涨20%，涨更多。结果人家都涨30%，不送外
看起来里面不差钱呀
超过了8000没
平均算下来
7000多
好的
有啥面试的心得吗
可以分享给其他小伙伴的
1面试前拿到offer，心态会不一样，面试时不用紧张，可以正常发挥，或者

卷王逆袭成功案例 武汉6年喜收4个优质offer 最高的年薪35W

2月9日改简历 4月15日晒offer

面试法宝:
改简历 + 实操

尼恩老师，新年好!
能帮忙修改下简历吗?
金三银四准备啦
可以的
java开发-6年-简历-...
拜托了，尼恩，希望能拿25K回来给你报喜
好，我加一下
还有吗?
恩恩，决赛圈offer了
截图是我自认能写出来的评分了，麻烦帮我参考下
选择大于努力，尼恩助我上岸
这么多offer，我看看哈
都是尼恩指点有方，本来还有个新能源汽车的，35W给拒了，主要太远了
跟着尼恩的时间太短了，目前实力也只能到这儿了
这里边有个大数据的，感觉也不错

卷王逆袭成功案例 6年小伙喜提4个Offer 最高涨9k，年薪35W

4月14日改简历 5月17日晒offer

涨薪法宝:
改简历 + 狠卷

Java开发工程师...
你看着我给你的
好的呀
好的呀
谢谢大佬
麻烦大佬了
这个你自己刷
不对的，你自己刷
那我照着这个改一下库存系统呀
一个简历，...
这么漂亮的简历，涨50%，已经没啥问题
只要准备好，不出大错，基本没问题啦
保证押金收起来，你的offer最高涨了9k，多返现100
后面继续跟着你，感觉卷的时间...
谢谢
我准备这一年的时间都看呢
加油卷哈
感觉自己学的不太透彻了
嗯哪
跟着大佬一起
我周围好几个年薪百万的，都是这

卷王逆袭成功案例

非全日制 6年经验卷王

喜提3个Offer，年包30W

5月9日改简历

5月18日晒offer

16:22 卷王

OK

最后一个项目的简历还没写好

写好了发你是吗?

那你写好发我我把

嗯嗯好的

那我先优化下

5月9日 下午16:22

高级 Java 开发工程师_严玉华-6年.docx

OK, 我先看看, 然后联系你哈

估计要几天

好的

面试法宝:
改简历 + 狠狠卷

16:21 卷王

是呀, 有点学到你这个回到问题的思路了, 你这个面试回到我前面说面试官真的就

这个行情看百度有几个博客面试题已经搞不定面试官了, 太搞了

是的, 现在面试, 都要面试官

催面试官就占便宜

不催就吃亏

对了, 你一共拿了几个 offer 呀

回头我打码, 就显一下

面试试了4家, 搞定了三家

这个牛逼了

OK

是呀

感谢面试官的这一波老秀的简历

加油卷哈

感谢面试官, 后面继续努力卷, 争取早日拿到架构去

再卷3年, 不差几个OK啦

卷王逆袭成功案例

4年卷王入职微软，涨50%

3月7日改简历 **5月12日晒offer**

微信聊天截图：

3月7日 上午9:17
感谢
3月7日 上午9:17
应星老师，什么时候有空 麻烦帮我看看简历哈

3月8日 下午6:01
收到10号了

3月8日 下午6:01
好嘛，早知道我早点找你了哈哈

3月8日 下午6:01
去年跟你说的外企最后没去，还是决定春招找更好的机会

3月8日 下午6:01
改一份小时间多，很耗时间，一天4封哦

3月8日 下午6:01
OK

3月8日 下午6:01
**涨薪法宝：
改简历 + 狠狠卷**

3月8日 下午6:01
面试邀约吗？
OK

3月8日 下午6:01
薪资涨幅如何？

3月8日 下午6:01
我之前的14k，现在年包28w，虽然薪资不如一线互联网大厂，但是福利还有工作都是挺好的

3月8日 下午6:01
在无锡

3月8日 下午6:01
那就好，这个不比大一大差

3月8日 下午6:01
月薪20k

3月8日 下午6:01
还是感谢您的书和视频帮助哈

3月8日 下午6:01
也差不多涨了个50%，现在的环境下，非常不错啦

3月8日 下午6:01
以后我还是会继续跟着看的

卷王逆袭成功案例

4年卷王入收2个offer，涨50%

3月23日改简历

5月12日晒offer

小伙高中学历 薪酬涨120%

5月6日改简历

7月22日晒offer

5月6日 14:03 周二

VIP7 康康 - 16666666666

5月6日 14:27:02

你这快估计要进你 668

¥668.00
5月6日 14:27:02

模块的开发工作，就这三个哈

某何师兄：[输入框]

哈哈，不用了上座，你啥你进了就行，光今天晚上辅导就值几千了

老弟很感谢您

还有很多其他的模块

面试官问

就是其他人做的

嘿嘿，好

7月22日 14:03 周二

VIP7 康康 - 16666666666

奈何老弟真的没有学历

问啥答啥

就是这工资

之前你的工资是多少来的

翻了一翻

我待返你多少得金来的

不知道，原价给你多少就行了

¥668.00
5月6日 14:27:02

咱们得说还算话呀

老弟拿那个那个高并发改造点，所向披靡。

ok

从头到尾面试官问的明明白白

后面继续聊聊哈

秘诀：
改简历 + 狠狠卷

5年卷王喜收2大Offer
最高涨5K

5月19日改简历 9月13日晒offer

秘诀:
改简历 + 狠狠卷

发我吧

辛苦了

OK, 简历还需好好改改哈

找主要介绍思路, 方法

嗯嗯, 我先消化一下。有问题在请教您

OK

12:20

您好, 我跳槽成功了, 目前手上有两家相对心仪的公司, 想您, 有空盼您参考下呗

恭喜恭喜🎉

一家是互联网, 主做云计算相关的, 给了23K, 一家是医疗, 做医疗器械的, 给22K

自研的医疗器械公司说

薪资涨幅都不多, 涨幅都不高, 都没达到你涨幅标准

感谢

好的, 稍等哈

简历: 简历 202205.docx
48.0 KB

微信电脑端

辛苦您了

目前薪资是18K, 目标想冲击到24K

OK, 我晚点看看回复哈

好的感谢

5月21日 周二 20:26

恩恩, 今天有空看一看我的简历嘛? 帮优化优化

后面跟着我在腾讯一道

卷王逆袭成功案例

双非二本小伙春招大翻身

喜提9大offer

2月22日改简历 4月13日晒offer

H VIP 85 运营岗 H VIP 85 运营岗

OK，稍晚一点站 面试法宝：
改简历+IM实操

9大offer 最高年薪30万

公司	部门	岗位	薪资结构	总包
1. 美团	能源数字化产品部	java后端开发	18.5w+14.5k+5k+200元/月+500股期权 <30w>	22.4w
2. 贝壳	交易研发部	前端	16w+14k	>22w
3. 快手	特定	java游戏开发	15w+5k+餐补+100元/月	14.3w
4. 字节跳动		全栈	13k+13	>16w
5. 小红书			14w	
6. 爱奇艺			9k+包住+租房津贴	
7. 腾讯广告			13w+餐补+班车	17w
8. 京东			14w	
9. 饿了么			14w	

修改简历找尼恩（资深简历优化专家）

- 如果面试表达不好，尼恩会提供 简历优化指导
- 如果项目没有亮点，尼恩会提供 项目亮点指导
- 如果面试表达不好，尼恩会提供 面试表达指导

作为 40 岁老架构师，尼恩长期承担技术面试官的角色：

- 从业以来，“阅历”无数，对简历有着点石成金、改头换面、脱胎换骨的指导能力。
- 尼恩指导过刚刚就业的小白，也指导过 P8 级的老专家，都指导他们上岸。

如何联系尼恩。尼恩微信，请参考下面的地址：

语雀：<https://www.yuque.com/crazymakercircle/gkkw8s/khigna>

码云：<https://gitee.com/crazymaker/SimpleCrayIM/blob/master/疯狂创客圈总目录.md>